

W4111 – Introduction to Databases

Module II (6), Big Data



Contents

Contents

- Course update
- Module II
 - Transactions
 - Concepts
 - Atomicity, durability and recovery
 - Isolation, concurrency control and locking
 - Scalability and high availability overview
- “Big Data”
 - Concepts
 - OLAP
 - MapReduce
 - Algebraic operations
- Conclusions

Course Update

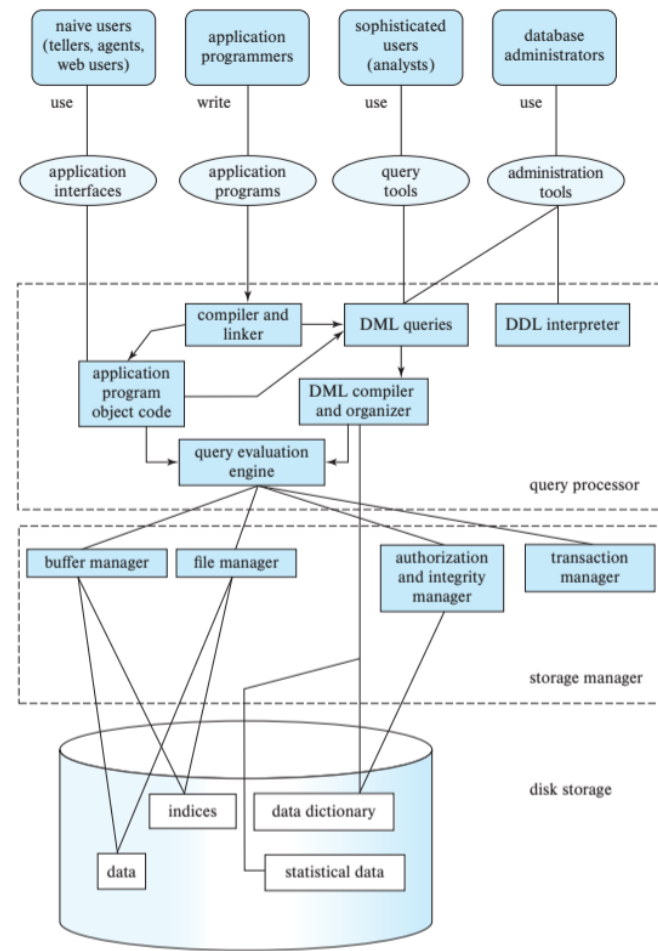
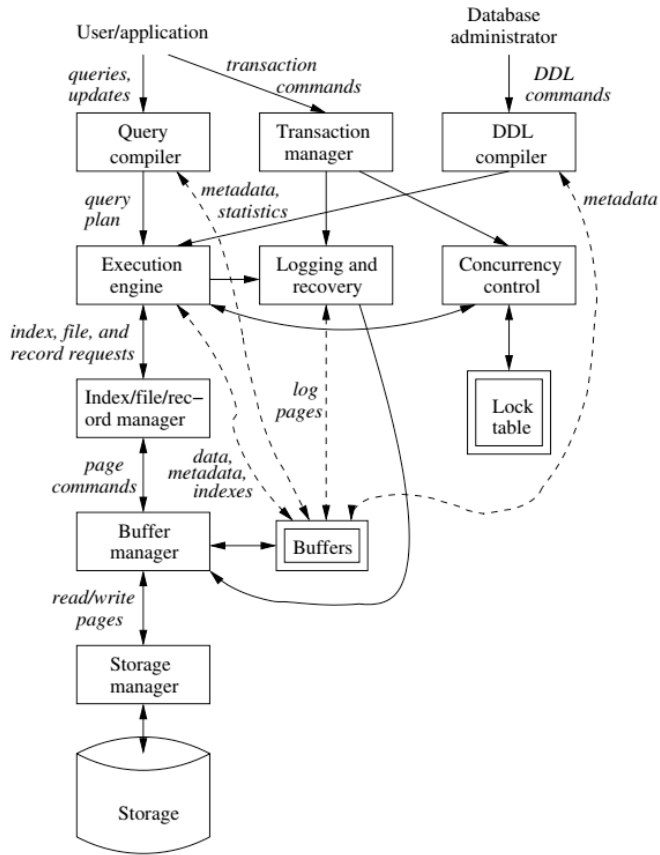
Course Update

- Exam 3 – Discussion

Module II – DBMS Architecture and Implementation

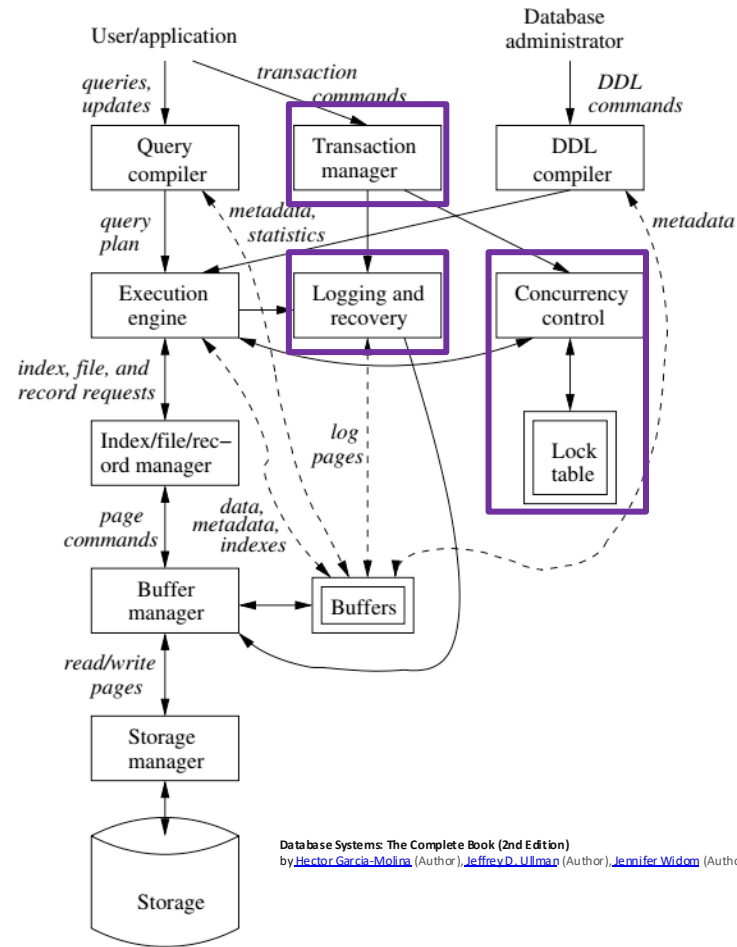
Module II – DBMS Architecture and Implementation Overview and Reminder

DBMS Arch.



Data Management

Today's topics.

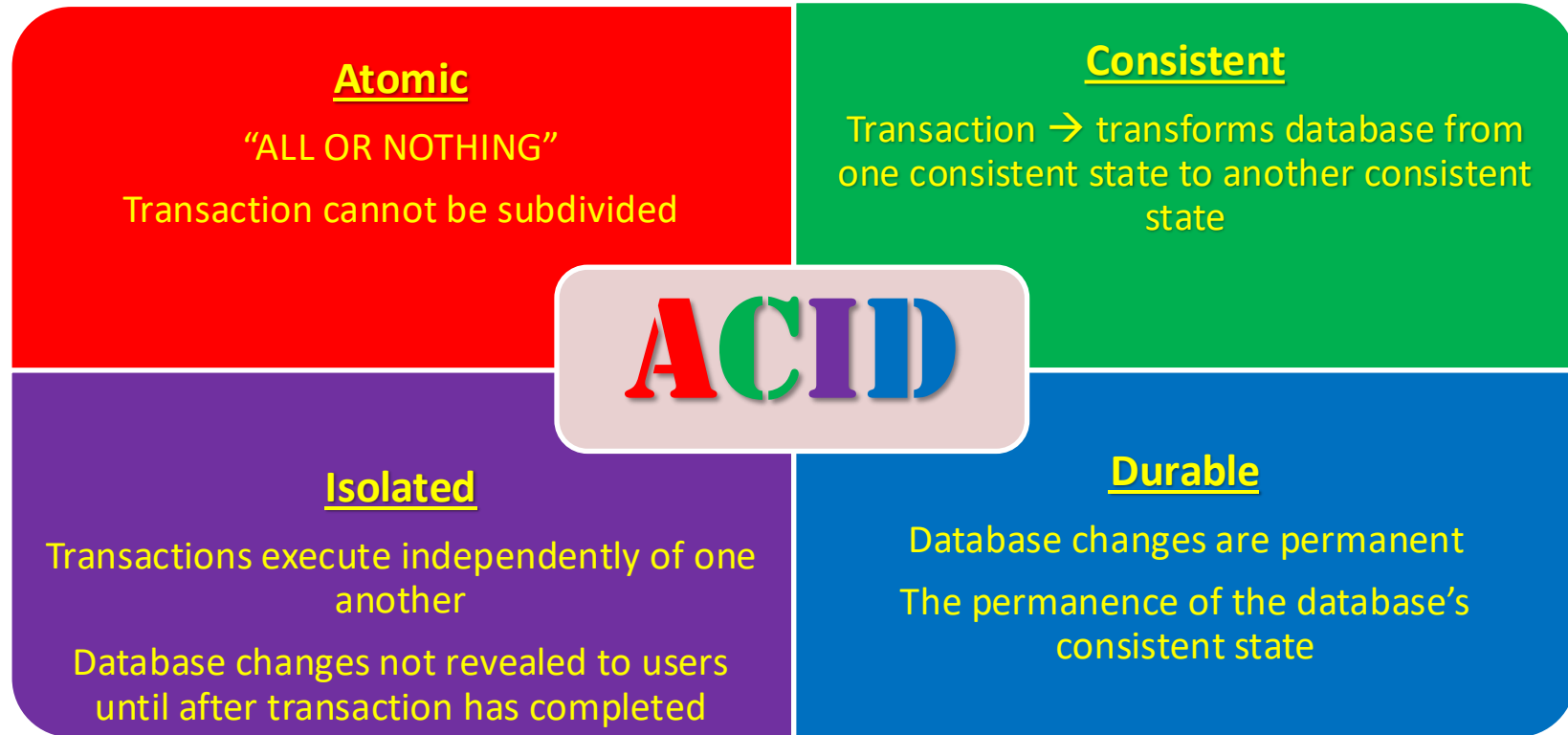


Transactions and Recovery

Core Concepts

Core Transaction Concept is ACID Properties

<http://slideplayer.com/slide/9307681>



ACID Properties (httpwww.tutorialspoint.com/dbms/dbms_transaction.htm)

A *transaction* is a very small unit of a program and it may contain several low-level tasks. A transaction in a database system must maintain **A**tomicity, **C**onsistency, **I**solation, and **D**urability – commonly known as ACID properties – in order to ensure accuracy, completeness, and data integrity.

- **Atomicity** – This property states that a transaction must be treated as an atomic unit, that is, either all of its operations are executed or none. There must be no state in a database where a transaction is left partially completed. States should be defined either before the execution of the transaction or after the execution/abortion/failure of the transaction.
- **Consistency** – The database must remain in a consistent state after any transaction. No transaction should have any adverse effect on the data residing in the database. If the database was in a consistent state before the execution of a transaction, it must remain consistent after the execution of the transaction as well.
- **Durability** – The database should be durable enough to hold all its latest updates even if the system fails or restarts. If a transaction updates a chunk of data in a database and commits, then the database will hold the modified data. If a transaction commits but the system fails before the data could be written on to the disk, then that data will be updated once the system springs back into action.
- **Isolation** – In a database system where more than one transaction are being executed simultaneously and in parallel, the property of isolation states that all the transactions will be carried out and executed as if it is the only transaction in the system. No transaction will affect the existence of any other transaction.

Transaction Concept

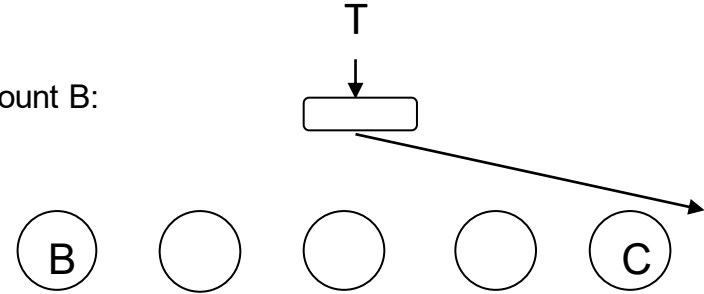
- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- E.g., transaction to transfer \$50 from account A to account B:
BEGIN TRANSACTION {
1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)
COMMIT or ROLLBACK }
Transfer(to_a, from_b, amount)
- Two main issues to deal with:
 - Failures of various kinds, such as hardware failures and system crashes
 - Concurrent execution of multiple transactions



Example of Fund Transfer

- Transaction to transfer \$50 from account A to account B:

1. **read**(A)
2. $A := A - 50$
3. **write**(A)
4. **read**(B)
5. $B := B + 50$
6. **write**(B)



- Atomicity requirement**

- If the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
 - Failure could be due to software or hardware
- The system should ensure that updates of a partially executed transaction are not reflected in the database

- Durability requirement** — once the user has been notified that the transaction has completed (i.e., the transfer of the \$50 has taken place), the updates to the database by the transaction must persist even if there are software or hardware failures.



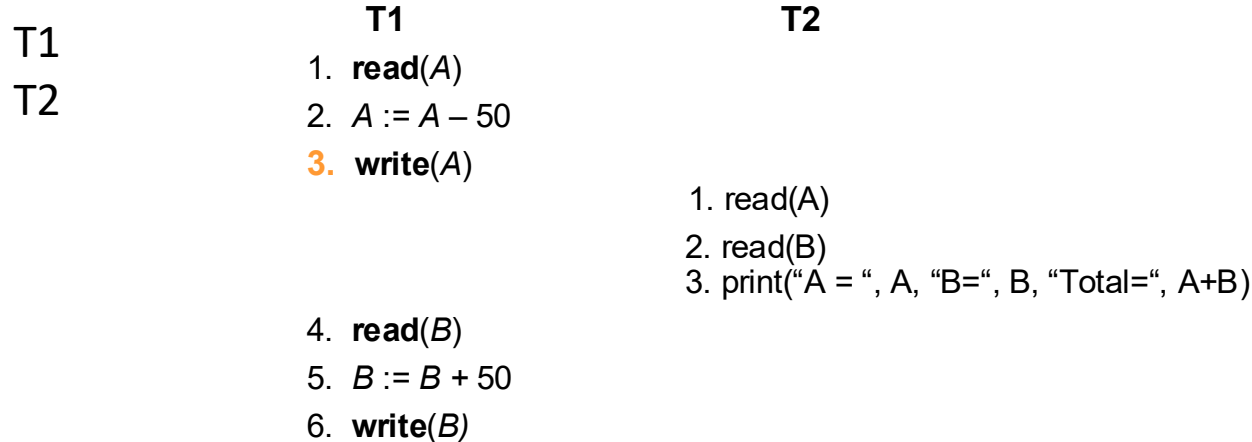
Example of Fund Transfer (Cont.)

- **Consistency requirement** in above example:
 - The sum of A and B is unchanged by the execution of the transaction
- In general, consistency requirements include
 - Explicitly specified integrity constraints such as primary keys and foreign keys
 - Implicit integrity constraints
 - e.g., sum of balances of all accounts, minus sum of loan amounts must equal value of cash-in-hand
 - A transaction must see a consistent database.
 - During transaction execution the database may be temporarily inconsistent.
 - When the transaction completes successfully the database must be consistent
 - Erroneous transaction logic can lead to inconsistency



Example of Fund Transfer (Cont.)

- T1, T2
- **Isolation requirement** — if between steps 3 and 6, another transaction T2 is allowed to access the partially updated database, it will see an inconsistent database (the sum $A + B$ will be less than it should be).



- Isolation can be ensured trivially by running transactions **serially**
 - That is, one after the other.
- However, executing multiple transactions concurrently has significant benefits, as we will see later.



ACID Properties

A **transaction** is a unit of program execution that accesses and possibly updates various data items. To preserve the integrity of data the database system must ensure:

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Execution of a transaction in isolation preserves the consistency of the database.
- **Isolation.** Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
 - That is, for every pair of transactions T_i and T_j , it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished.
- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.



Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
 - Restart the transaction
 - Can be done only if no internal logical error
 - Kill the transaction
- **Committed** – after successful completion.

Atomicity Durability

Atomicity

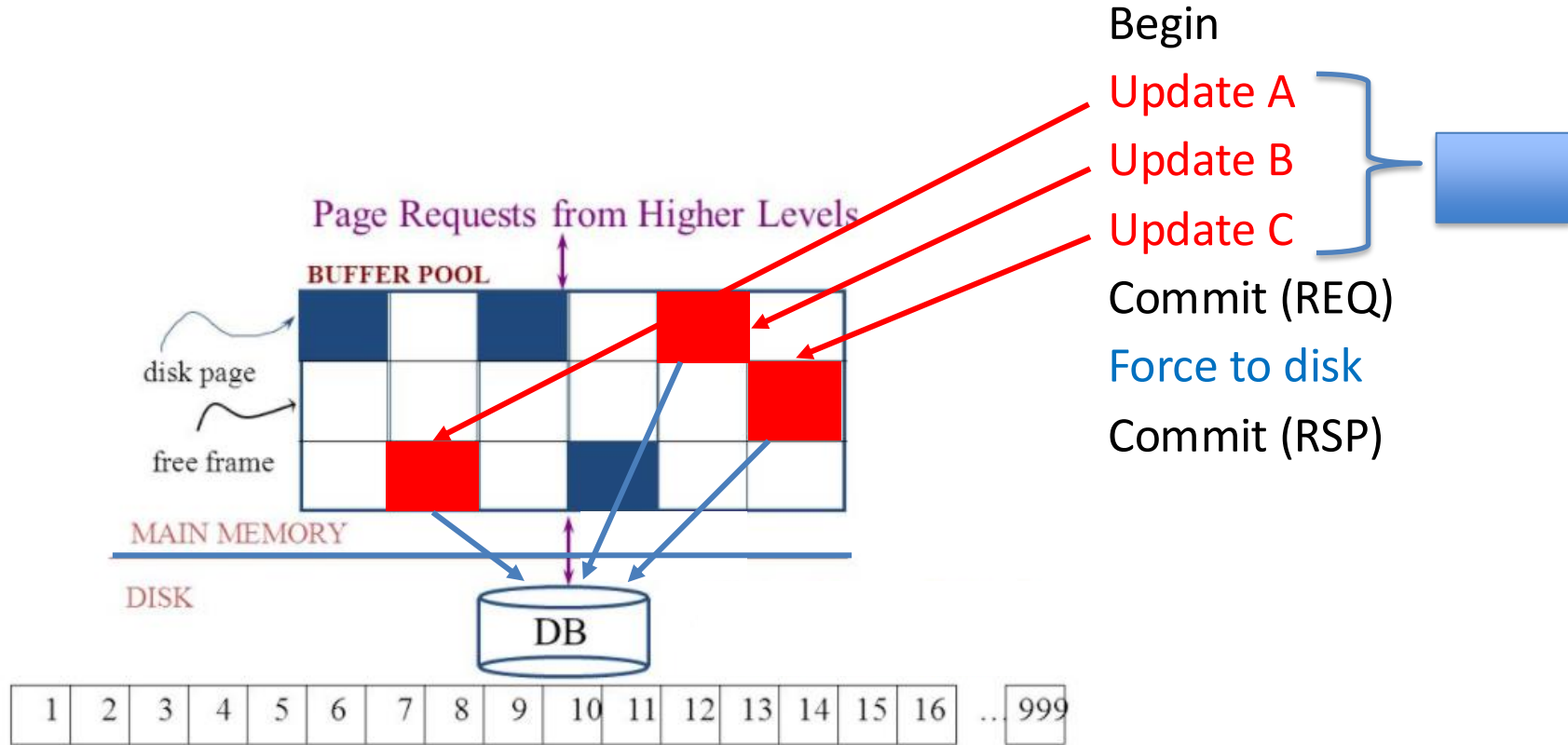
A transaction is a logical unit of work that must be either entirely completed or entirely undone. (All writes happen or none of them happen)

OK. What does this mean? Consider some pseudo-code

```
def transfer(source_acct_id, target_acct_id, amount)
```

1. Check that both accounts exist.
2. IF *is_checking_account(source_acct_id)*
 1. Check that $(\text{source_acct.balance} - \text{amount}) > \text{source_account.overdraft_limit}$
3. ELSE
 1. Check that $(\text{source_count.balance} - \text{amount}) > \text{source_account.minimum_balance}$
4. Update source account
5. Update target account.
6. INSERT a record into transfer tracking table.

Simplistic Approach



Simplistic Approach

There are several problems with the simplistic approach.

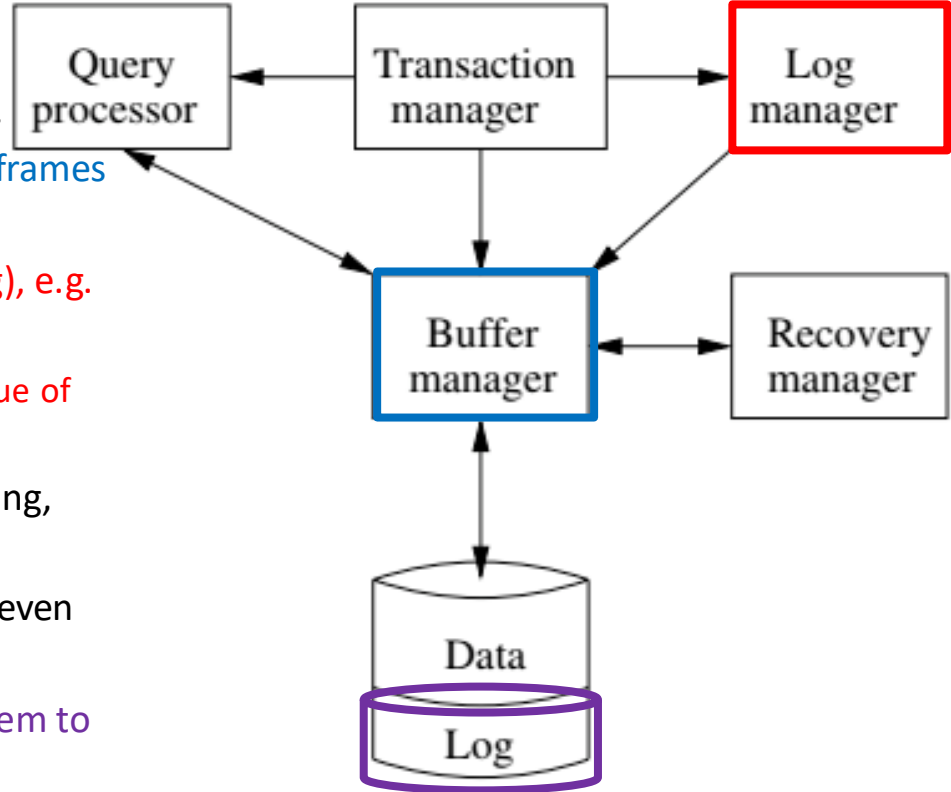
1. The approach does not solve the problem
 1. Some writes might succeed.
 2. Some might be interrupted by the failure or require retry.
2. Writes may be random and scattered. N updates might
 1. Change a few bytes in N data frames
 2. A few bytes in M index frames

Transaction rate becomes bottlenecked by write I/O rate, even though a relatively small number of bytes change/transaction.
3. Written frames must be held in memory.
 1. Lots of transactions
 2. Randomly writing small pieces of lots of frames.
 3. Consumes lots of memory with pinned pages.
 4. Degrades the performance and optimization of the buffer.
 1. The optimal buffer replacement policy wants to hold frames that will be reused.
 2. Not frames that have been touched and never reused.

DBMS ACID Implementation

Implementation Subsystems

- *Query processor* schedules and executes queries.
- *Buffer manager* controls reading/writing blocks/frames to/from disk.
- *Log manager* journals/records events to disk (log), e.g.
 - Transaction start/commit/abort
 - Transaction update of a block and previous value of data.
- *Transaction manager* coordinates query scheduling, buffer read/write and logging to ensure ACID.
- *Recovery manager* processes log to ensure ACID even after transaction or system failures.
- *Log* is a special type of block file used by the system to optimize performance and ensure ACID.

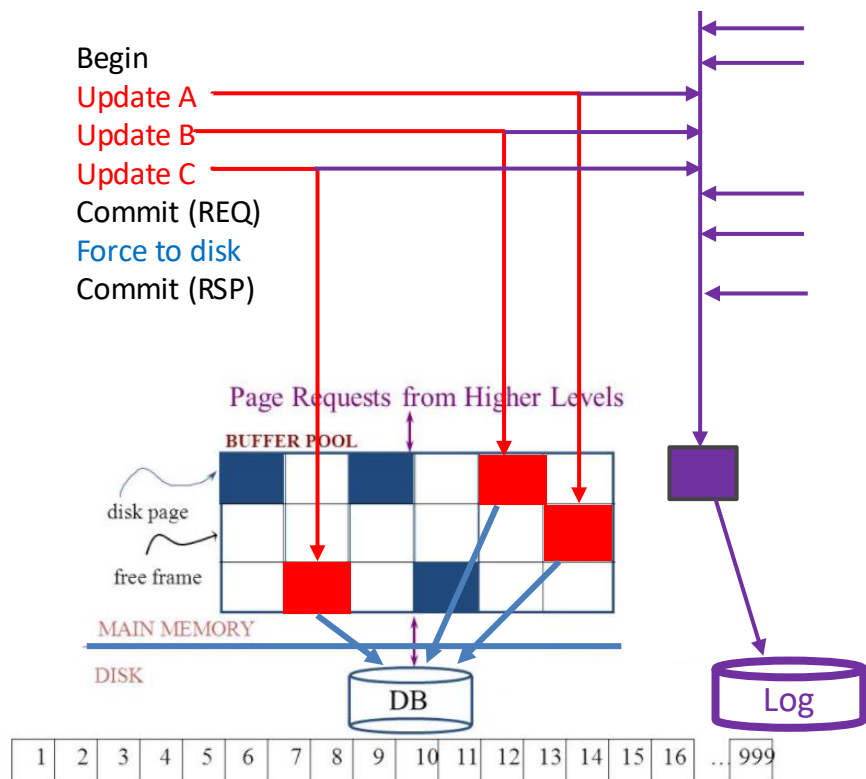


Logging

The DBMS logs every transaction event

- *Log Sequence Number (LSN)*: A unique ID for a log record.
- *Prev LSN*: A link to their last log record.
- *Transaction ID number*.
- *Type*: Describes the type of database log record.
 - **Update Log Record**
 - *PageID*: A reference to the Page ID of the modified page.
 - *Length and Offset*: Length in bytes and offset of the page are usually included.
 - *Before and After Images* of records.
 - **Compensation Log Record**
 - **Commit Record**
 - **Abort Record Checkpoint Record**
 - **Completion Record** notes that all work has been done for this particular transaction.

Write Ahead Logging



DBMS (Redo processing)

- Write log events from all transactions into a single log stream.
- Multiple events per page
- Forces (writes) log record on COMMIT/ABORT
 - Single block I/O records many updates
 - Versus multiple block I/Os, each recording a single change.
 - All of a transaction's updates recorded in one I/O versus many.
- If there is a failure
 - DBMS sequentially reads log.
 - Applies changes to modified pages that were not saved to disk.
 - Then resumes normal processing.

Write Ahead Logging

- Force every write to disk?
 - Poor response time.
 - But provides durability.
- Steal buffer-pool frames from uncommitted transactions?
 - If not, poor performance/caching performance
 - If yes, how can we ensure atomicity?
Uncommitted updates on disk

	No Steal	Steal
Force	Trivial	
No Force		Desired

DBMS (Undo processing)

- Enable steal policy to improve cache performance by
 - Avoiding lots of pinned pages
 - Unlikely to be reused soon.
- Before stealing
 - Force log record to disk.
 - Update log entry has data record
 - Before image
 - After image
- If there is a failure
 - DBMS sequentially reads log.
 - Undoes changes to
 - modified pages, uncommitted pages
 - That were saved to disk.
 - Then resumes normal processing.

ARIES recovery involves three passes

1. Analysis pass:

- Determine which transactions to undo
- Determine which pages were dirty (disk version not up to date) at time of
- RedoLSN: LSN from which redo should start

2. Redo pass:

- Repeats history, redoing all actions from RedoLSN
(updated committed but not written changes to pages)
- RecLSN and PageLSNs are used to avoid redoing actions already reflected on page

3. Undo pass:

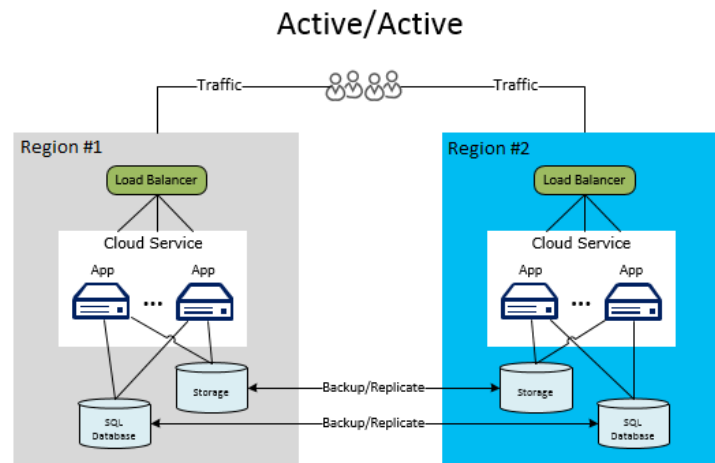
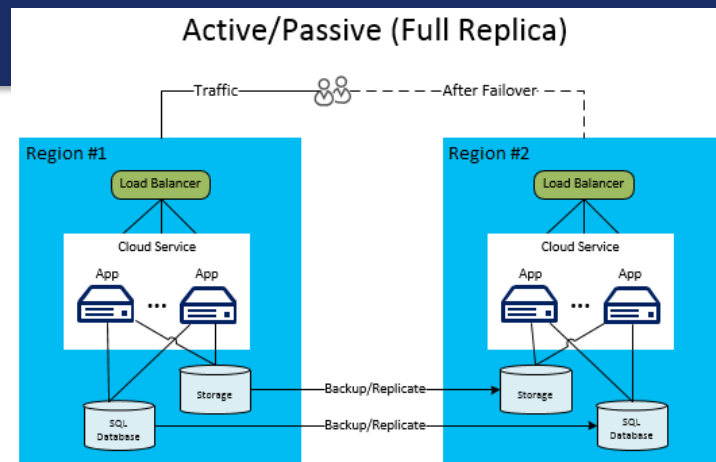
- Rolls back all incomplete transactions (with uncommitted pages written to disk).
- Transactions whose abort was complete earlier are not undone

Durability

- Write changes to disk trivially achieves durability.
- DBMS engine uses write-ahead-logging to
 - Achieve durability
 - But with better performance through more efficient caching and I/O.
- Well, disks fail. How is that durable.
 - RAID and other solutions.
 - Disk subsystems, including entire RAID device, fail →
 - Duplex writes
 - To independent disk subsystems.
- Well, there are earthquakes, floods, etc.

Availability and Replication

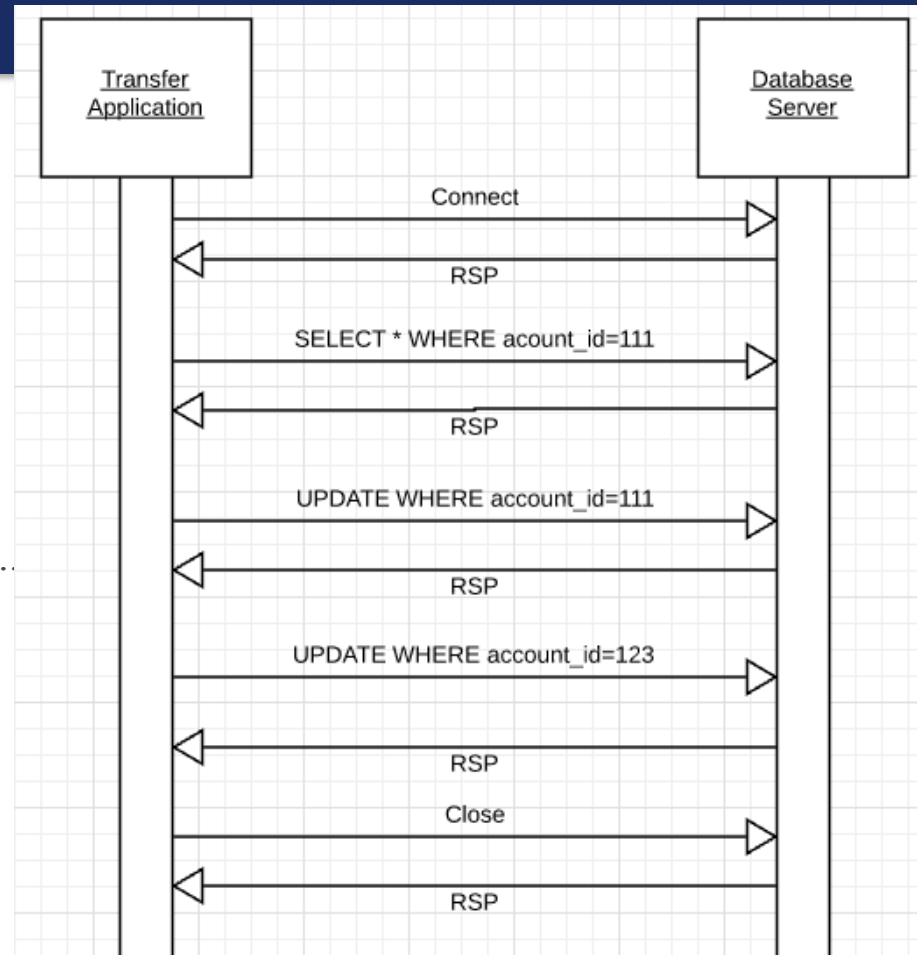
- There are two basic patterns
 - Active/Passive
 - All requests go to *master* during normal processing.
 - Updates are transactionally queued for processing at passive backup.
 - Failure of *master*
 - Routes subsequent requests to *backup*.
 - Backup must process and commit updates before accepting requests.
 - Active/Active
 - Both environments process requests.
 - Some form of distributed transaction commit required to synchronize updates on both copies.
- Multi-system communication to guarantee consistency is the foundation for tradeoffs in CAP.
 - The system can be CAP if and only iff
 - There are never any partitions or system failures
 - Which is unrealistic in cloud/Internet systems.



Isolation

Isolation

- Transfer \$50 from
 - account_id=111 to
 - account_id=123
- Requires 3 SQL statements
 - SELECT from 111 to check balance $\geq$ \$50
 - UPDATE account_id=111
 - UPDATE account_id=123
- There are some interesting scenarios
 - Two different programs read the balance (\$51)
 - And decide removing \$50 is OK.
- DB constraints can prevent the conflict from happening, but ...
 - There are more complex scenarios that constraints do not prevent.
 - Not ALL databases support constraints.
 - The “correct” execution should be that
 - One transaction responds “insufficient funds”
 - Before attempting transfer instead of after attempting.



Isolation

- Consider two simultaneous transfer transactions T1 and T2.
- There are two equally **correct** executions
 1. T1 transfers, T2 responds “insufficient funds” and does not attempt transfer
 2. T2 transfers, T1 responds “insufficient funds” and does not attempt transfer
- Each correct simultaneous execution is equivalent to a serial (sequential) execution schedule
 - (1) Execute T1, Execute T2
 - (2) Execute T2, Execute T1
- NOTE:
 - We are focusing on correctness not
 - Fairness:
 - We do not care which transaction was actually submitted first.
 - And probably do not know due to networking, etc.

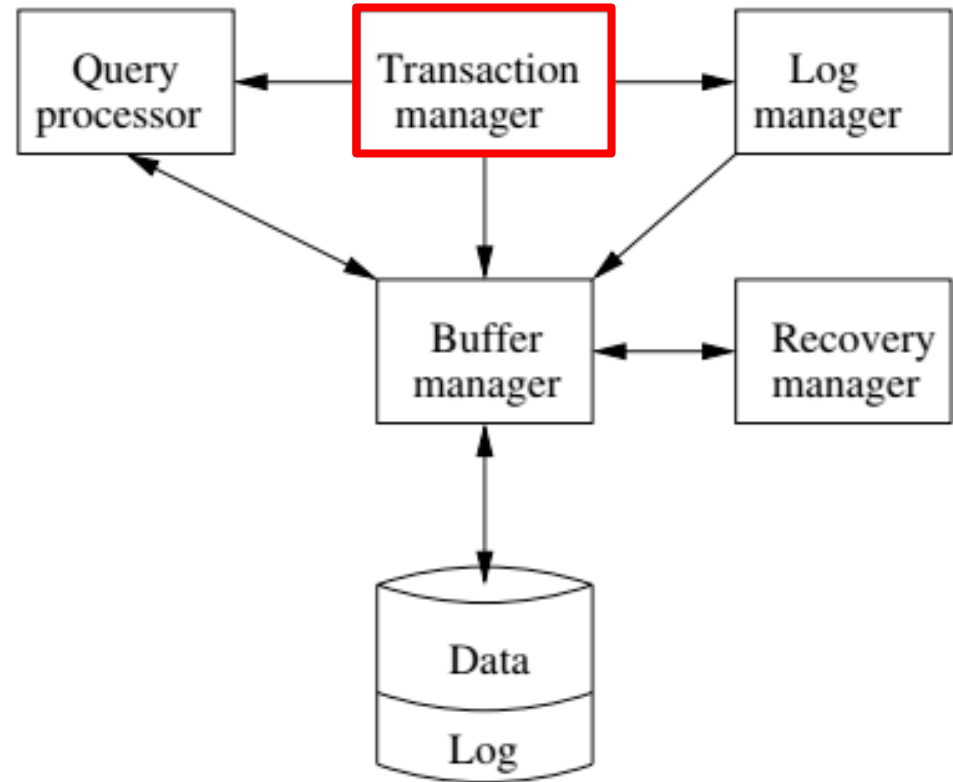
Serializability

“In [concurrency control](#) of [databases](#),^{[1][2]} [transaction processing](#) (transaction management), and various [transactional](#) applications (e.g., [transactional memory](#)^[3] and [software transactional memory](#)), both centralized and [distributed](#), a transaction [schedule](#) is **serializable** if its outcome (e.g., the resulting database state) is equal to the outcome of its transactions executed serially, i.e. without overlapping in time. Transactions are normally executed concurrently (they overlap), since this is the most efficient way. Serializability is the major correctness criterion for concurrent transactions' executions. It is considered the highest level of [isolation](#) between [transactions](#), and plays an essential role in [concurrency control](#). As such it is supported in all general purpose database systems.”
(<https://en.wikipedia.org/wiki/Serializability>)

DBMS ACID Implementation

Implementation Subsystems

- *Query processor* schedules and executes queries.
- *Buffer manager* controls reading/writing blocks/frames to/from disk.
- *Log manager* journals/records events to disk (log), e.g.
 - Transaction start/commit/abort
 - Transaction update of a block and previous value of data.
- *Transaction manager* coordinates query **scheduling**, buffer read/write and logging to ensure ACID.
- *Recovery manager* processes log to ensure ACID even after transaction or system failures.
- *Log* is a special type of block file used by the system to optimize performance and ensure ACID.



Garcia-Molina et al., p. 846

Schedule

18.1.1 Schedules

A *schedule* is a sequence of the important actions taken by one or more transactions. When studying concurrency control, the important read and write actions take place in the main-memory buffers, not the disk. That is, a database element A that is brought to a buffer by some transaction T may be read or written in that buffer not only by T but by other transactions that access A .

T_1	T_2
READ(A, t)	READ(A, s)
$t := t+100$	$s := s*2$
WRITE(A, t)	WRITE(A, s)
READ(B, t)	READ(B, s)
$t := t+100$	$s := s*2$
WRITE(B, t)	WRITE(B, s)

Figure 18.2: Two transactions

Garcia-Molina et al.

18.1.2 Serial Schedules

- Assume there are three
 - concurrently executing transactions
 - T1, T2 and T3
- The transaction manager
 - Enables concurrent execution
 - But schedules individual operations
 - To ensure that the final DB state
 - Is *equivalent* to one of the following schedules
 - T1, T2, T3
 - T1, T3, T2
 - T2, T1, T3
 - T2, T3, T1
 - T3, T1, T2
 - T3, T2, T1

Concurrent execution was *serializable*.

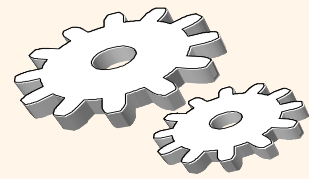
A schedule is *serial* if its actions consist of all the actions of one transaction, then all the actions of another transaction, and so on. No mixing of the actions is allowed.

T_1	T_2	A	B
		25	25
READ(A,t)			
t := t+100			
WRITE(A,t)		125	
READ(B,t)			
t := t+100			
WRITE(B,t)			125
	READ(A,s)		
	s := s*2		
	WRITE(A,s)	250	
	READ(B,s)		
	s := s*2		
	WRITE(B,s)		250

Figure 18.3: Serial schedule in which T_1 precedes T_2

Serializability (en.wikipedia.org/wiki/Serializability)

- **Serializability** is used to keep the data in the data item in a consistent state. Serializability is a property of a transaction [schedule](#) (history). It relates to the [isolation](#) property of a [database transaction](#).
- **Serializability** of a schedule means equivalence (in the outcome, the database state, data values) to a *serial schedule* (i.e., sequential with no transaction overlap in time) with the same transactions. It is the major criterion for the correctness of concurrent transactions' schedule, and thus supported in all general purpose database systems.
- **The rationale behind serializability** is the following:
 - If each transaction is correct by itself, i.e., meets certain integrity conditions,
 - then a schedule that comprises any *serial* execution of these transactions is correct (its transactions still meet their conditions):
 - "Serial" means that transactions do not overlap in time and cannot interfere with each other, i.e, complete *isolation* between each other exists.
 - Any order of the transactions is legitimate, (...)
 - As a result, a schedule that comprises any execution (not necessarily serial) that is equivalent (in its outcome) to any serial execution of these transactions, is correct.



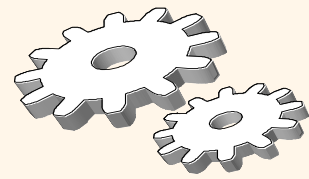
Lock-Based Concurrency Control

❖ Strict Two-phase Locking (Strict 2PL) Protocol:

- Each Xact must obtain a **S (shared) lock** on object before reading, and an **X (exclusive) lock** on object before writing.
- All locks held by a transaction are released when the transaction completes
 - **(Non-strict) 2PL Variant:** Release locks anytime, but cannot acquire locks after releasing any lock.
- If an Xact holds an X lock on an object, no other Xact can get a lock (S or X) on that object.

❖ Strict 2PL allows only serializable schedules.

- Additionally, it simplifies transaction aborts
- **(Non-strict) 2PL** also allows only serializable schedules, but involves more complex abort processing



Aborting a Transaction

- ❖ If a transaction T_i is aborted, all its actions have to be undone. Not only that, if T_j reads an object last written by T_i , T_j must be aborted as well!
- ❖ Most systems avoid such *cascading aborts* by releasing a transaction's locks only at commit time.
 - If T_i writes an object, T_j can read this only after T_i commits.
- ❖ In order to *undo* the actions of an aborted transaction, the DBMS maintains a *log* in which every write is recorded. This mechanism is also used to recover from system crashes: all active Xacts at the time of the crash are aborted when the system comes back up.

MySQL (Locking) Isolation

13.3.6 SET TRANSACTION Syntax

```
1  SET [GLOBAL | SESSION] TRANSACTION
2      transaction_characteristic [, transaction_characteristic] ...
3
4  transaction_characteristic:
5      ISOLATION LEVEL level
6      | READ WRITE
7      | READ ONLY
8
9  level:
10     REPEATABLE READ
11     | READ COMMITTED
12     | READ UNCOMMITTED
13     | SERIALIZABLE
```

Scope of Transaction Characteristics

You can set transaction characteristics globally, for the current session, or for the next transaction:

- With the `GLOBAL` keyword, the statement applies globally for all subsequent sessions. Existing sessions are unaffected.
- With the `SESSION` keyword, the statement applies to all subsequent transactions performed within the current session.
- Without any `SESSION` or `GLOBAL` keyword, the statement applies to the next (not started) transaction performed within the current session. Subsequent transactions revert to using the `SESSION` isolation level.

Isolation Levels

([https://en.wikipedia.org/wiki/Isolation_\(database_systems\)](https://en.wikipedia.org/wiki/Isolation_(database_systems)))

Not all transaction use cases require 2PL and serializable execution. Databases support a set of levels.

- **Serializable**

- With a lock-based [concurrency control](#) DBMS implementation, [serializability](#) requires read and write locks (acquired on selected data) to be released at the end of the transaction. Also *range-locks* must be acquired when a [SELECT](#) query uses a ranged *WHERE* clause, especially to avoid the [phantom reads](#) phenomenon.
- *The execution of concurrent SQL-transactions at isolation level SERIALIZABLE is guaranteed to be serializable. A serializable execution is defined to be an execution of the operations of concurrently executing SQL-transactions that produces the same effect as some serial execution of those same SQL-transactions. A serial execution is one in which each SQL-transaction executes to completion before the next SQL-transaction begins.*

- **Repeatable reads**

- In this isolation level, a lock-based [concurrency control](#) DBMS implementation keeps read and write locks (acquired on selected data) until the end of the transaction. However, *range-locks* are not managed, so [phantom reads](#) can occur.
- Write skew is possible at this isolation level, a phenomenon where two writes are allowed to the same column(s) in a table by two different writers (who have previously read the columns they are updating), resulting in the column having data that is a mix of the two transactions.^{[3][4]}

- **Read committed**

- In this isolation level, a lock-based [concurrency control](#) DBMS implementation keeps write locks (acquired on selected data) until the end of the transaction, but read locks are released as soon as the [SELECT](#) operation is performed (so the [non-repeatable reads phenomenon](#) can occur in this isolation level). As in the previous level, *range-locks* are not managed.
- Putting it in simpler words, read committed is an isolation level that guarantees that any data read is committed at the moment it is read. It simply restricts the reader from seeing any intermediate, 'dirty' read. It makes no promise whatsoever that if the transaction re-issues the read, it will find the same data; data is free to change after it is read.

- **Read uncommitted**

- This is the *lowest* isolation level. In this level, [dirty reads](#) are allowed, so one transaction may see *not-yet-committed* changes made by other transactions

In Databases, Cursors Define *Isolation*

- We have talked about ACID transactions

Isolation level	Dirty reads	Non-repeatable reads	Phantoms
Read Uncommitted	may occur	may occur	may occur
Read Committed	-	may occur	may occur
Repeatable Read	-	-	may occur
Serializable	-	-	-

```
InitialContext ctx = new InitialContext();
DataSource ds = (DataSource)
ctx.lookup("jdbc/MyBase");
Connection con = ds.getConnection();
DatabaseMetaData dbmd = con.getMetaData();
if (dbmd.supportsTransactionIsolationLevel(TRANSACTION_SERIALIZABLE)
{ Connection.setTransactionIsolation(TRANSACTION_SERIALIZABLE); }
```

- Isolation
 - Determines what happens when two or more threads are manipulating the data at the same time.
 - And is defined relative to where cursors are and what they have touched.
 - Because the cursor movement determines *what you are reading or have read*.
- *But, ... Cursors are client conversation state and cannot be used in REST.*

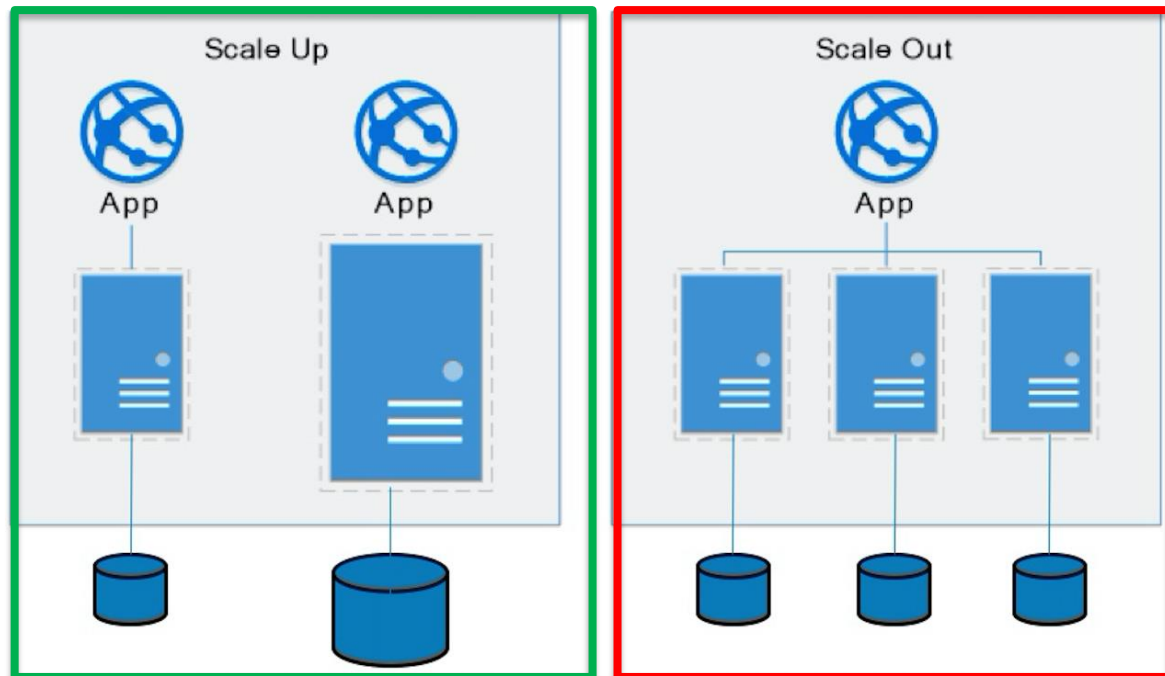
Scalability *Availability*

Approaches to Scalability

Scalability is the property of a system to handle a growing amount of work by adding resources to the system.

Replace system with a bigger machine,
e.g. more memory, CPU,

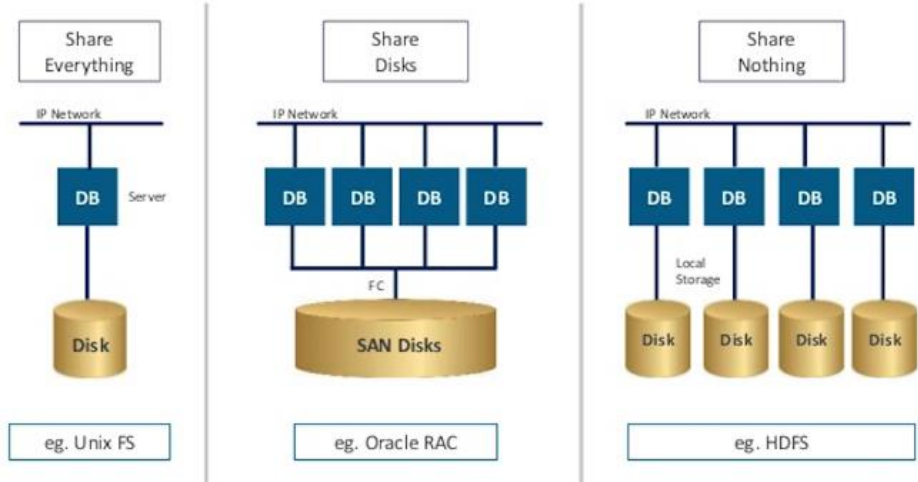
Add another system.



- Scale-up:
 - Less incremental.
 - More disruptive.
 - More expensive for extremely large systems.
 - Does not improve availability
- Scale-out:
 - Incremental cost.
 - Data replication enables availability.
 - Does not work well for functions like JOIN, referential integrity,

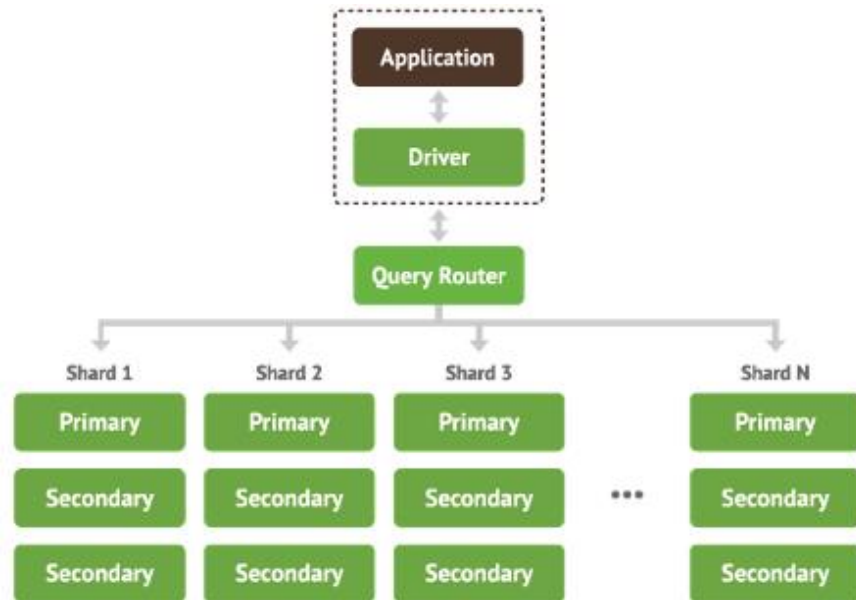
Disk Architecture for Scale-Out

- Share disks:
 - Is basically scale-up for data/disks. You can use NAS, SAN and RAID.
 - Isolation/Integrity requires distributed locking to control access from multiple database servers.
- Share nothing:
 - Is basically scale-out for disks.
 - Data is partitioned into *shards* based on a function $f()$ applied to a key.
 - Can improve availability, at the code consistency, with data replication.
 - There is a router that sends requests to the proper shard based on the function.

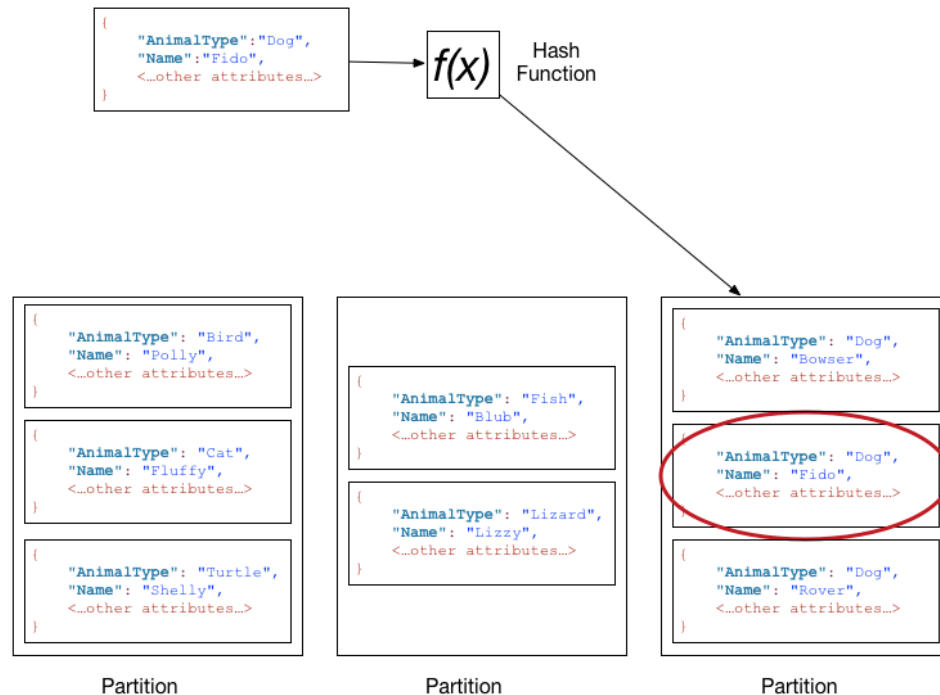


Shared Nothing, Scale-Out

MongoDB Sharding



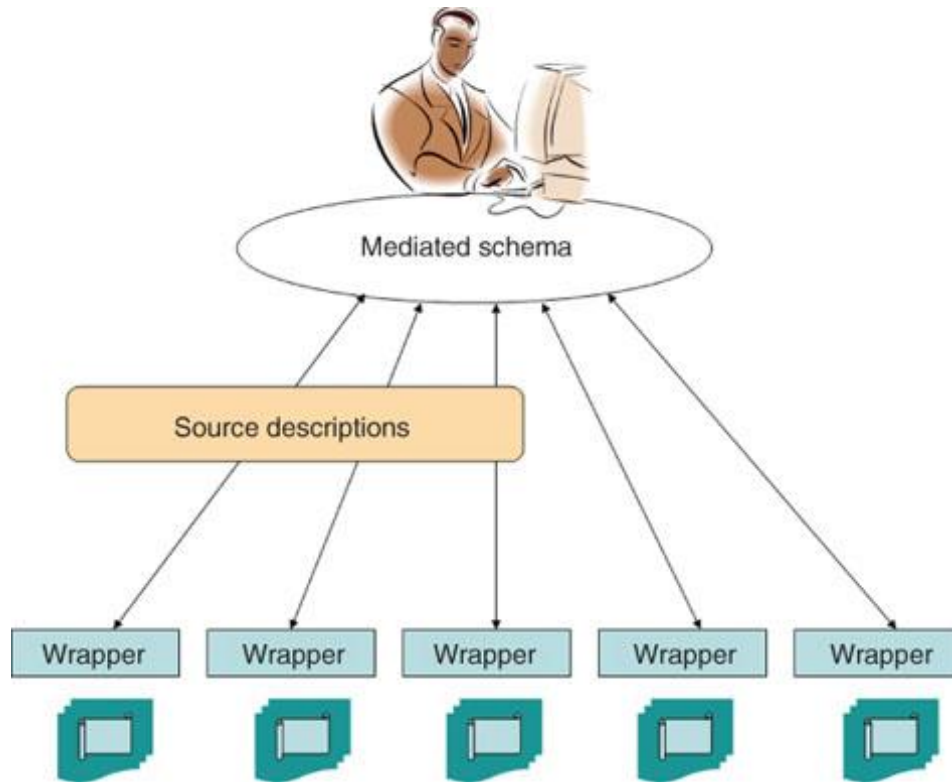
DynamoDB Partitioning



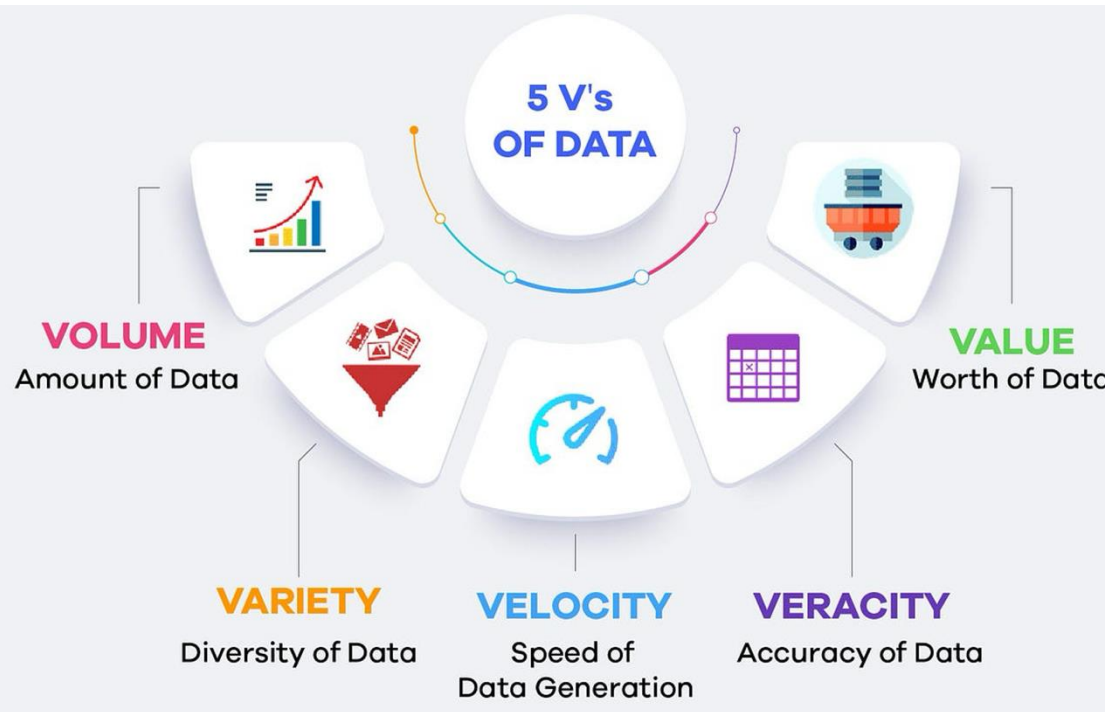
Big Data

Core Concepts

Enterprise Information Integration



5 V's of Big Data



https://medium.com/@get_excelsior/big-data-explained-the-5v-s-of-data-ae80cbe8ded1

Types Of Big Data

Following are the types of Big Data:

1. **Structured**
2. **Unstructured**
3. **Semi-structured**

Structured

Any data that can be stored, accessed and processed in the form of fixed format is termed as a 'structured' data. Over the period of time, talent in computer science has achieved greater success in developing techniques for working with such kind of data (where the format is well known in advance) and also deriving value out of it. However, nowadays, we are foreseeing issues when a size of such data grows to a huge extent, typical sizes are being in the range of multiple zettabytes.

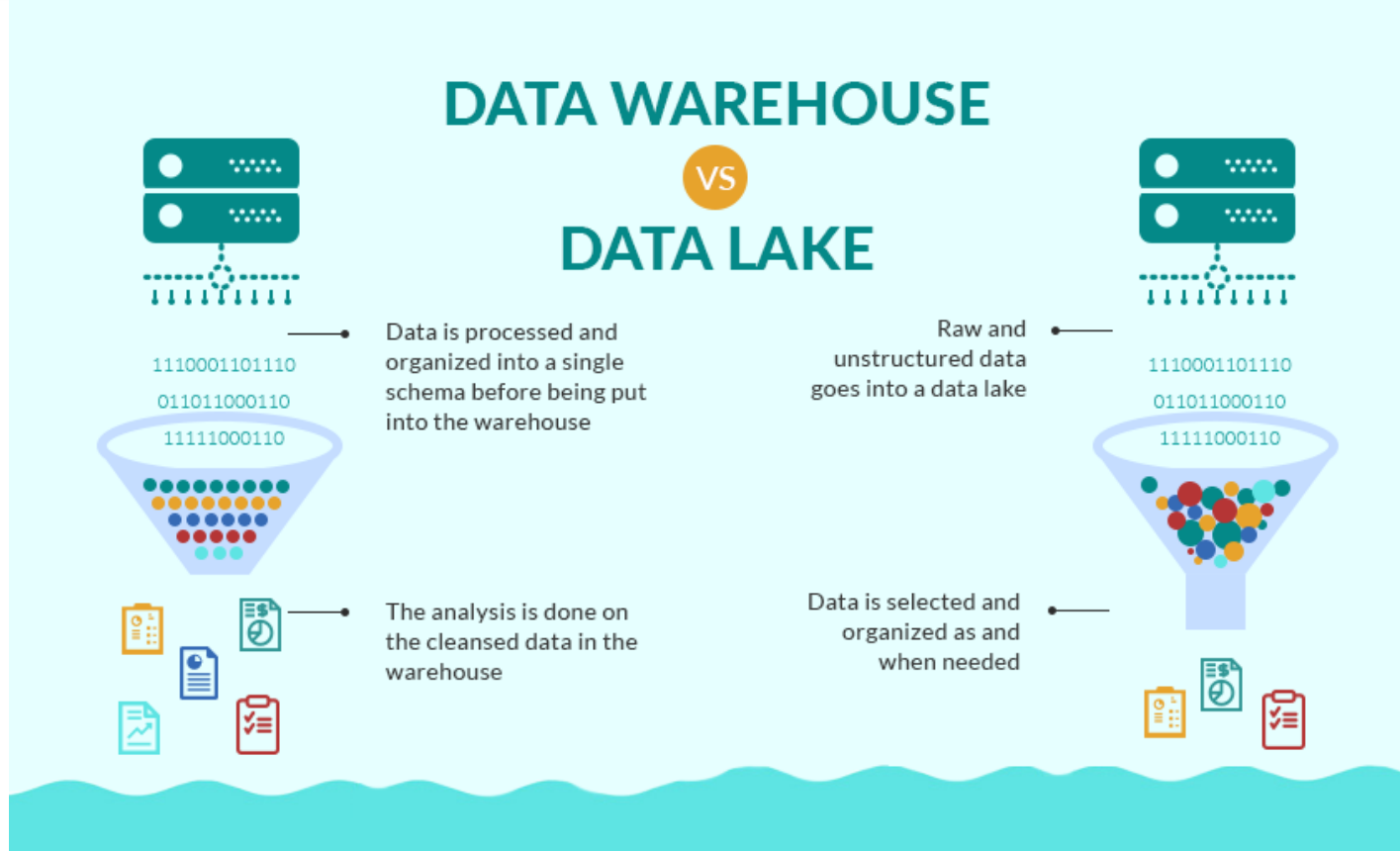
Unstructured

Any data with unknown form or the structure is classified as unstructured data. In addition to the size being huge, un-structured data poses multiple challenges in terms of its processing for deriving value out of it. A typical example of unstructured data is a heterogeneous data source containing a combination of simple text files, images, videos etc. Now day organizations have wealth of data available with them but unfortunately, they don't know how to derive value out of it since this data is in its raw form or unstructured format.

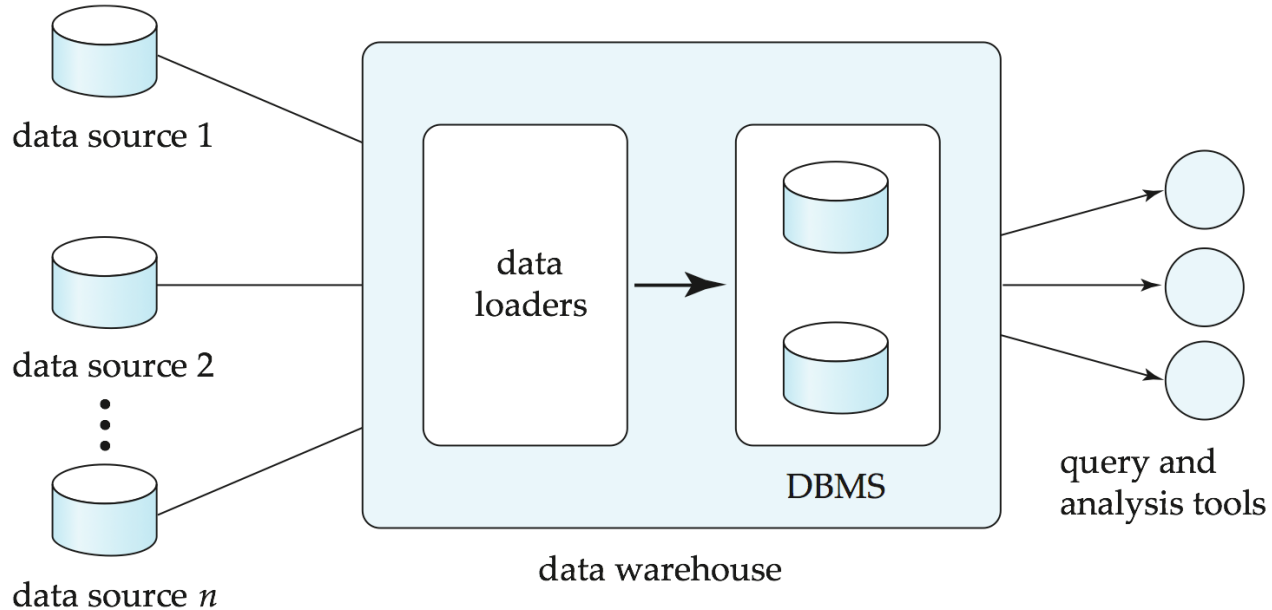
Semi-structured

Semi-structured data can contain both the forms of data. We can see semi-structured data as a structured in form but it is actually not defined with e.g. a table definition in relational **DBMS**. Example of semi-structured data is a data represented in an XML file.

Data Warehouse and Data Lake



Data Warehousing

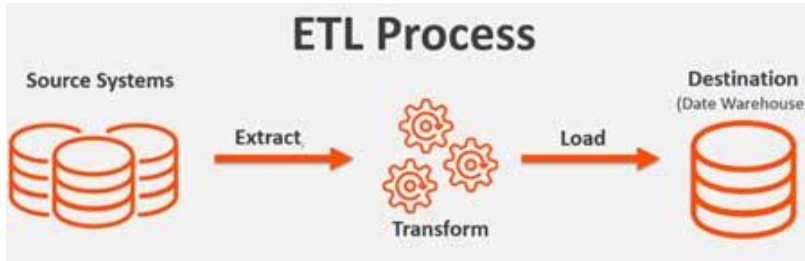


Overview (Cont.)

- Common steps in data analytics
 - Gather data from multiple sources into one location
 - Data warehouses also integrated data into common schema
 - Data often needs to be **extracted** from source formats, **transformed** to common schema, and **loaded** into the data warehouse
 - Can be done as **ETL (extract-transform-load)**, or **ELT (extract-load-transform)**
 - Generate aggregates and reports summarizing data
 - Dashboards showing graphical charts/reports
 - **Online analytical processing (OLAP) systems** allow interactive querying
 - Statistical analysis using tools such as R/SAS/SPSS
 - Including extensions for parallel processing of big data
 - Build **predictive models** and use the models for decision making

ETL Concepts

<https://databricks.com/glossary/extract-transform-load>



Extract

The first step of this process is extracting data from the target sources that could include an ERP, CRM, Streaming sources, and other enterprise systems as well as data from third-party sources. There are different ways to perform the extraction: **Three**

Data Extraction methods:

1. Partial Extraction – The easiest way to obtain the data is if the source system notifies you when a record has been changed
2. Partial Extraction- with update notification – Not all systems can provide a notification in case an update has taken place; however, they can point those records that have been changed and provide an extract of such records.
3. Full extract – There are certain systems that cannot identify which data has been changed at all. In this case, a full extract is the only possibility to extract the data out of the system. This method requires having a copy of the last extract in the same format so you can identify the changes that have been made.

Transform

Next, the transform function converts the raw data that has been extracted from the source server. As it cannot be used in its original form in this stage it gets cleansed, mapped and transformed, often to a specific data schema, so it will meet operational needs. This process entails several transformation types that ensure the quality and integrity of data; below are the most common as well as advanced transformation types that prepare data for analysis:

- Basic transformations:
 - Cleaning
 - Format revision
 - Data threshold validation checks
 - Restructuring
 - Deduplication
- Advanced transformations:
 - Filtering
 - Merging
 - Splitting
 - Derivation
 - Summarization
 - Integration
 - Aggregation
 - Complex data validation

Load

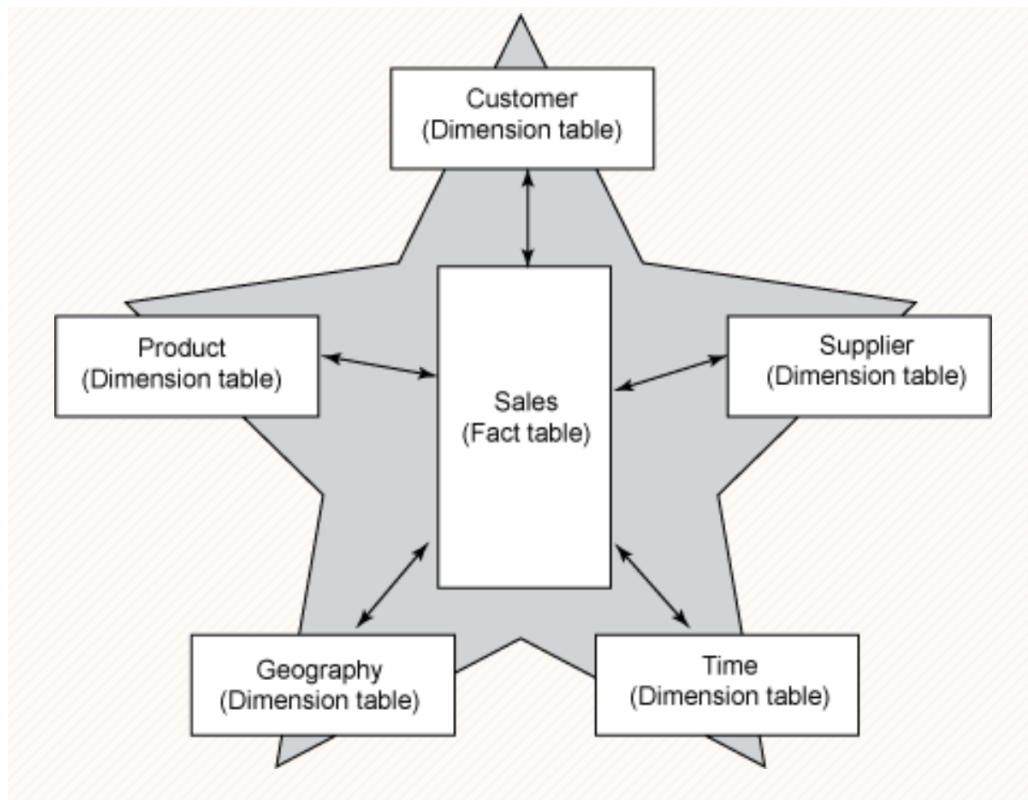
Finally, the load function is the process of writing converted data from a staging area to a target database, which may or may not have previously existed. Depending on the requirements of the application, this process may be either quite simple or intricate.

Star Schema



Facts and Dimensions

- A common model for (some) data in a data warehouse is a *star schema*.
- There are one or more *fact tables* that have records for small facts, e.g.
 - Sold product A.
 - To customer B.
 - At price C.
- Facts are positioned in dimensions, e.g.
 - Date, time
 - Location
 - Product category
 -
- Queries ask for summations and aggregations in dimensions, e.g.
 - Total sales of products in category X, to customers in country Y during year Z.
 - Queries can roll-up, drill down, pivot,
- This is a generalization of the spreadsheet concept of *pivot tables*.

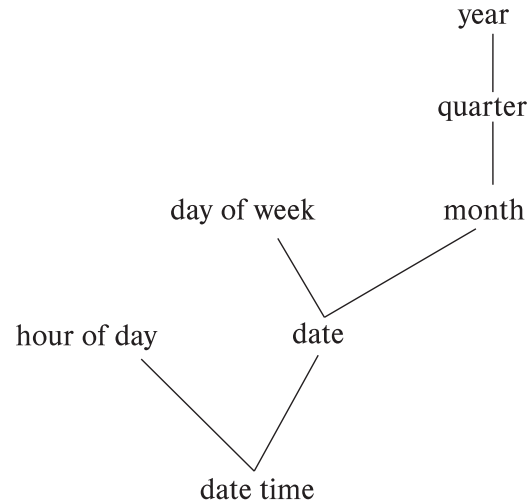
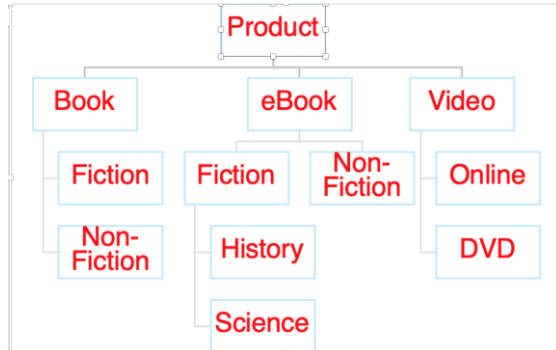




Hierarchies on Dimensions

- **Hierarchy** on dimension attributes: lets dimensions be viewed at different levels of detail
- E.g., the dimension *datetime* can be used to aggregate by hour of day, date, day of week, month, quarter or year

Another dimension could be ...
Product category.



(a) time hierarchy



(b) location hierarchy

OLAP



Data Analysis and OLAP

- **Online Analytical Processing (OLAP)**

- Interactive analysis of data, allowing data to be summarized and viewed in different ways in an online fashion (with negligible delay)

- We use the following relation to illustrate OLAP concepts

- *sales (item_name, color, clothes_size, quantity)*

This is a simplified version of the *sales* fact table joined with the dimension tables, and many attributes removed (and some renamed)

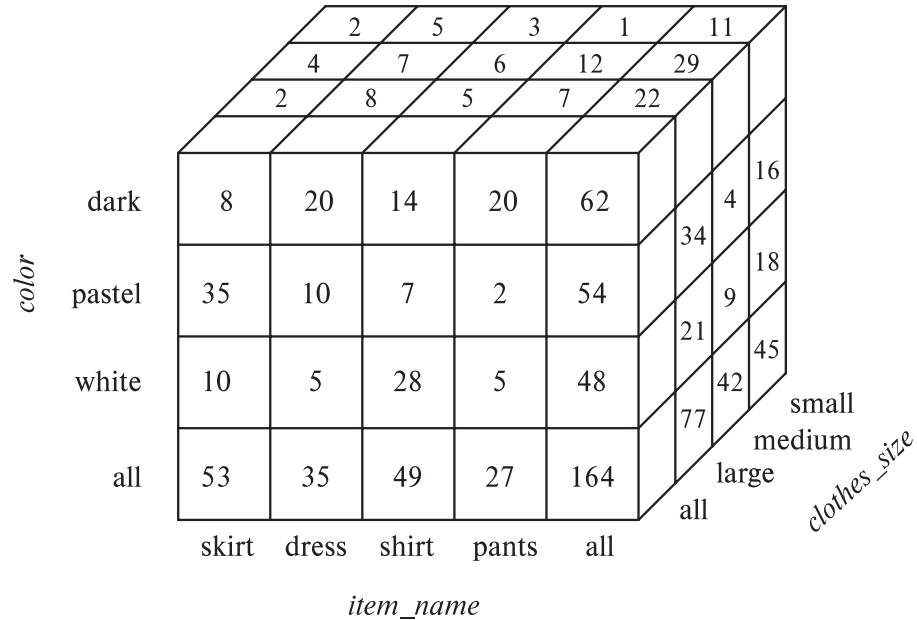
- For Classic Models, this would be:

- sales(productCode, quantityOrdered, priceEach)
- But, I need to link with dimensions.
 - Date
 - Location
 - ProductType
- ➔
- sales(productCode, quantityOrdered, priceEach, date_dimension_id, location_dimension_id, product_type_dimension_id)
- We are only going to worry about date and location for now.



Data Cube

- A **data cube** is a multidimensional generalization of a cross-tab
- Can have n dimensions; we show 3 below
- Cross-tabs can be used as views on a data cube





Online Analytical Processing Operations

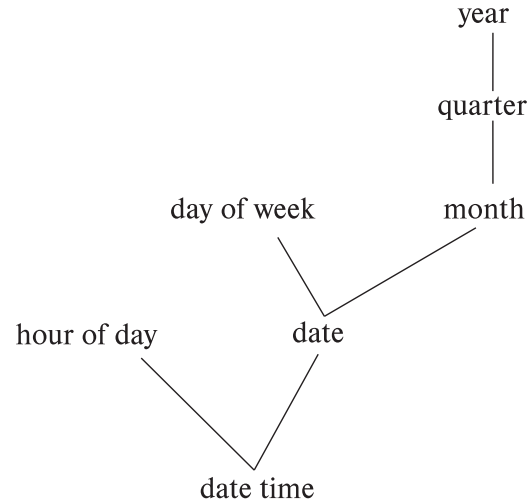
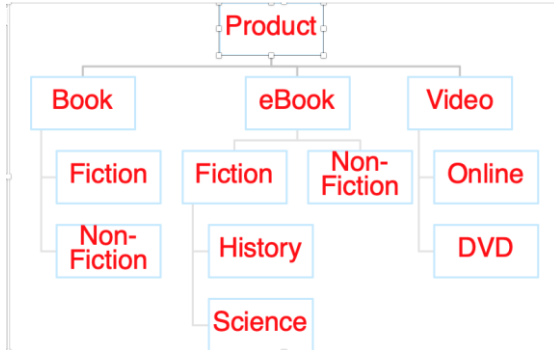
- **Pivoting:** changing the dimensions used in a cross-tab
 - E.g., moving colors to column names
- **Slicing:** creating a cross-tab for fixed values only
 - E.g., fixing color to white and size to small
 - Sometimes called **dicing**, particularly when values for multiple dimensions are fixed.
- **Rollup:** moving from finer-granularity data to a coarser granularity
 - E.g., aggregating away an attribute
 - E.g., moving from aggregates by day to aggregates by month or year
- **Drill down:** The opposite operation - that of moving from coarser-granularity data to finer-granularity data



Hierarchies on Dimensions

- **Hierarchy** on dimension attributes: lets dimensions be viewed at different levels of detail
- E.g., the dimension *datetime* can be used to aggregate by hour of day, date, day of week, month, quarter or year

Another dimension could be ...
Product category.



(a) time hierarchy



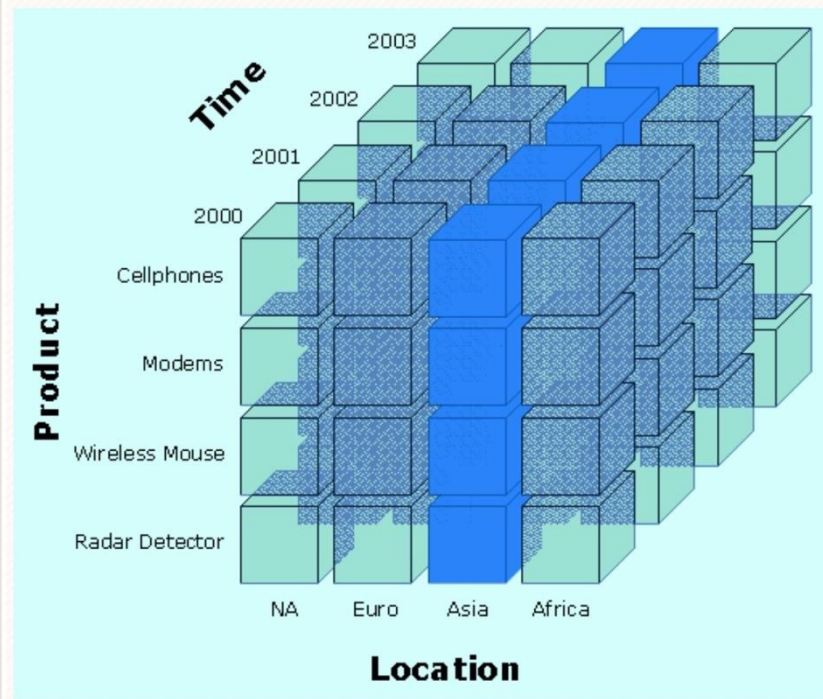
(b) location hierarchy



Slice

Slice:

A slice is a subset of a multi-dimensional array corresponding to a single value for one or more members of the dimensions not in the subset.

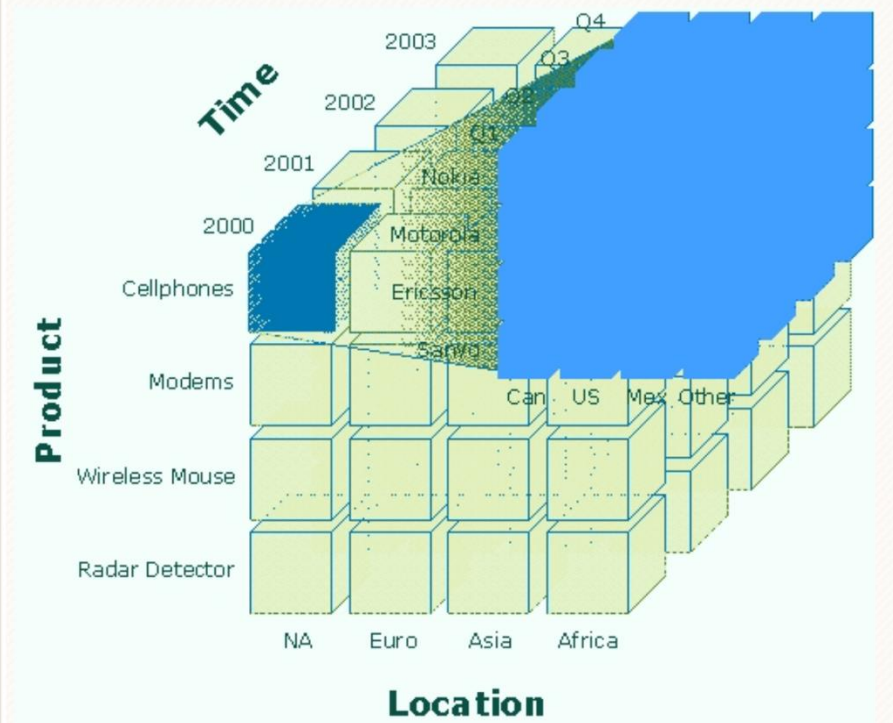




Dice

Dice:

The dice operation is a slice on more than two dimensions of a data cube (or more than two consecutive slices).

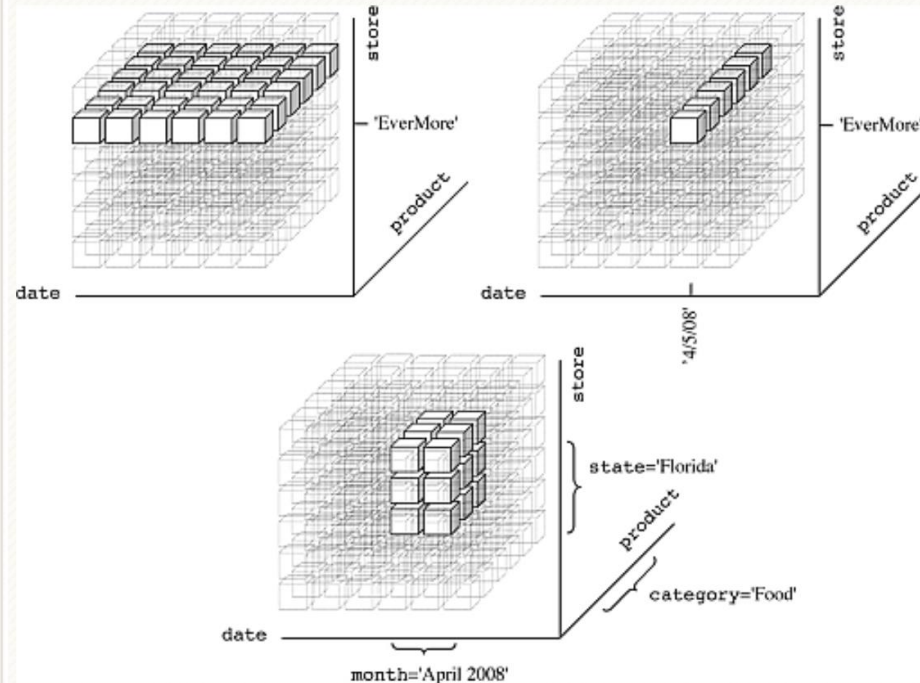




Drilling

Drill Down/Up:

Drilling down or up is a specific analytical technique whereby the user navigates among levels of data ranging from the most summarized (up) to the most detailed (down).

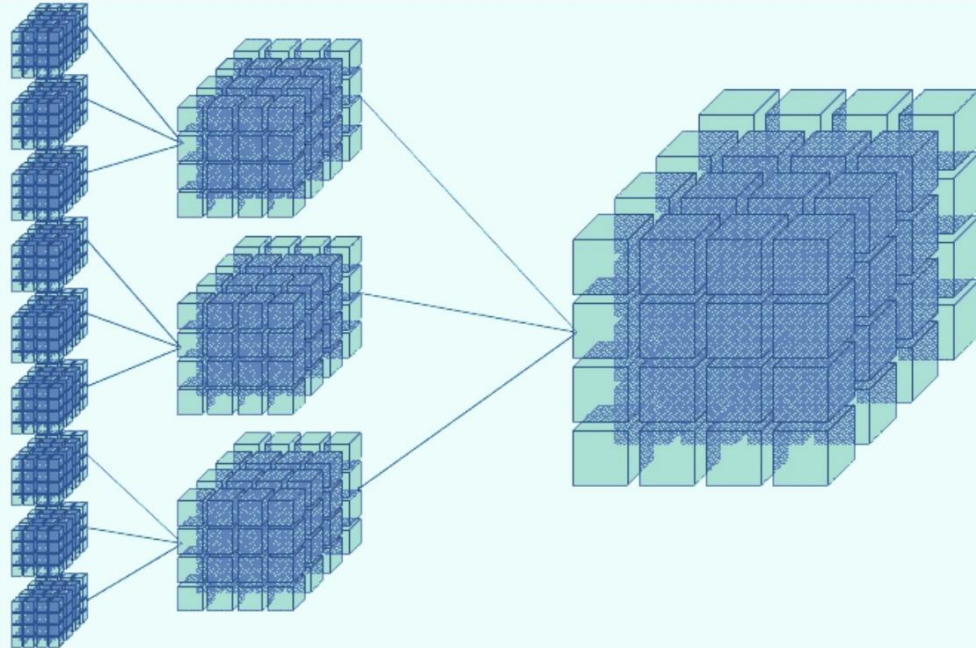




Roll-up

Roll-up:

(Aggregate, Consolidate) A roll-up involves computing all of the data relationships for one or more dimensions. To do this, a computational relationship or formula might be defined.

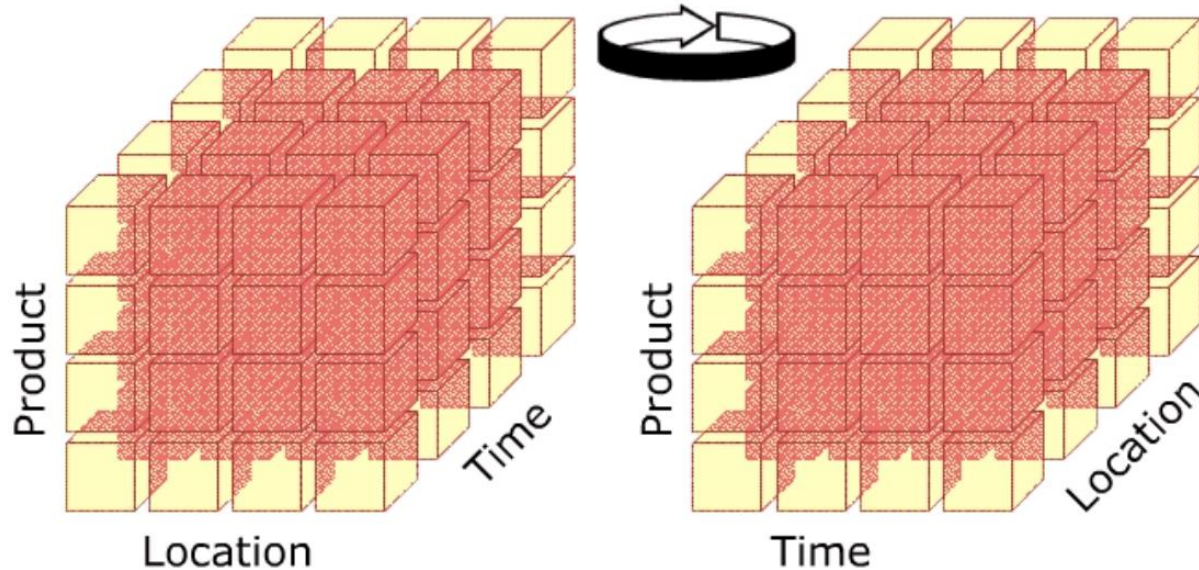




Pivot

Pivot:

This operation is also called rotate operation. It rotates the data in order to provide an alternative presentation of data – the report or page display takes a different dimensional orientation.

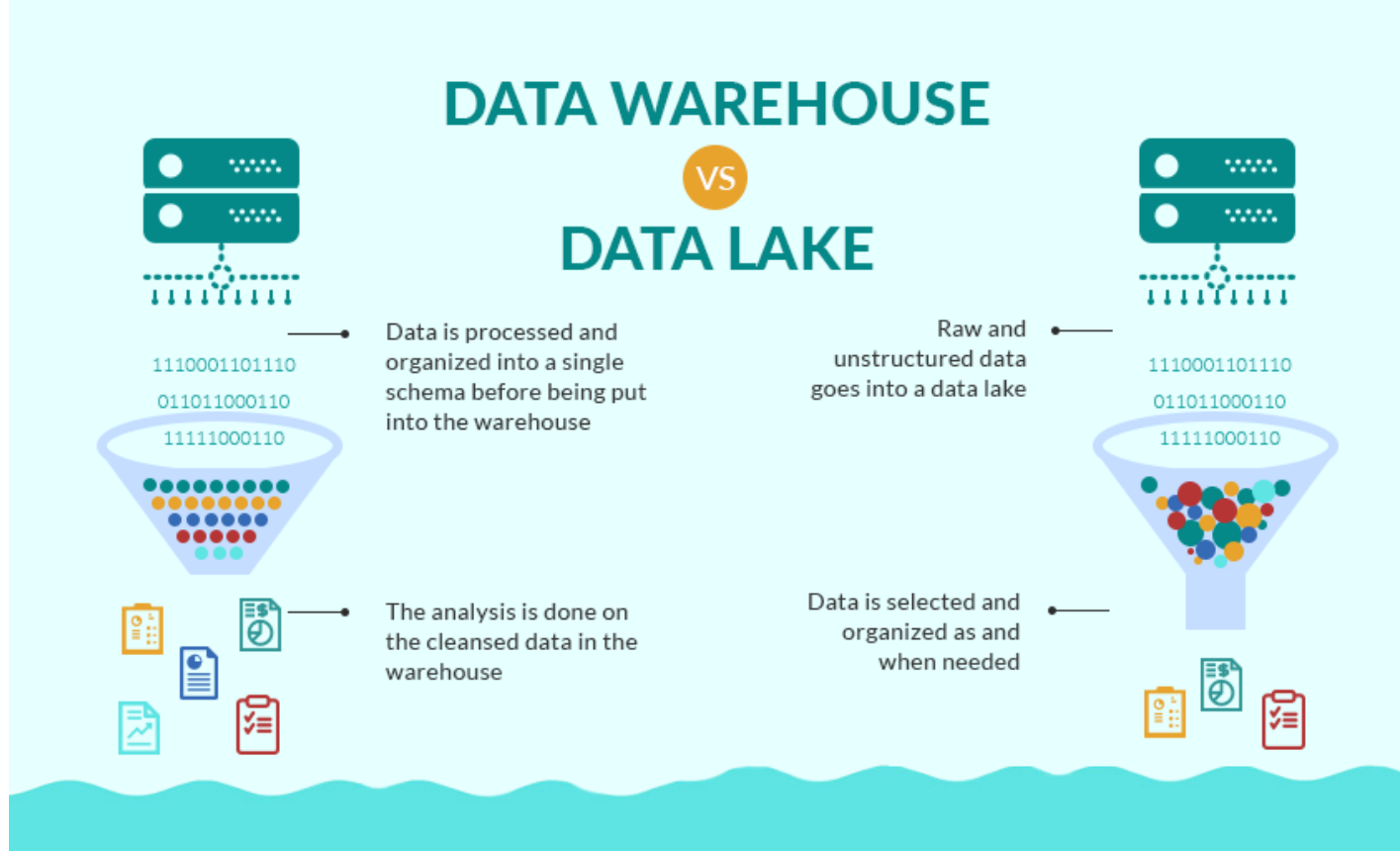


Summary

- OLAP is a very useful model for analyzing business data, typically data that is inherently tabular to begin with.
- There are pure OLAP databases, but a more common approach is:
 - Use a relational database.
 - Define a star schema.
 - ETL from the operational data into the star schema.
 - “Translate” the logical OLAP operations into SQL queries and predefined views.
- What is it that you really care about?
 - In the past, I had students build a star schema from Classic Models and do OLAP-like queries. You will not have OLAP on a HW.
 - For the final, you will be responsible for understanding the concepts. For example, it is very common for me to provide some schema/data and ask you to define a star schema and some hypothetical queries.

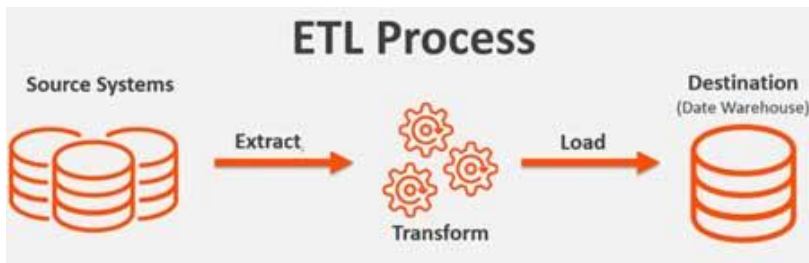
Data Engineering

Data Warehouse and Data Lake



ETL Concepts

<https://databricks.com/glossary/extract-transform-load>



Extract

The first step of this process is extracting data from the target sources that could include an ERP, CRM, Streaming sources, and other enterprise systems as well as data from third-party sources. There are different ways to perform the extraction: **Three**

Data Extraction methods:

1. Partial Extraction – The easiest way to obtain the data is if the source system notifies you when a record has been changed
2. Partial Extraction- with update notification – Not all systems can provide a notification in case an update has taken place; however, they can point those records that have been changed and provide an extract of such records.
3. Full extract – There are certain systems that cannot identify which data has been changed at all. In this case, a full extract is the only possibility to extract the data out of the system. This method requires having a copy of the last extract in the same format so you can identify the changes that have been made.

Transform

Next, the transform function converts the raw data that has been extracted from the source server. As it cannot be used in its original form in this stage it gets cleansed, mapped and transformed, often to a specific data schema, so it will meet operational needs. This process entails several transformation types that ensure the quality and integrity of data; below are the most common as well as advanced transformation types that prepare data for analysis:

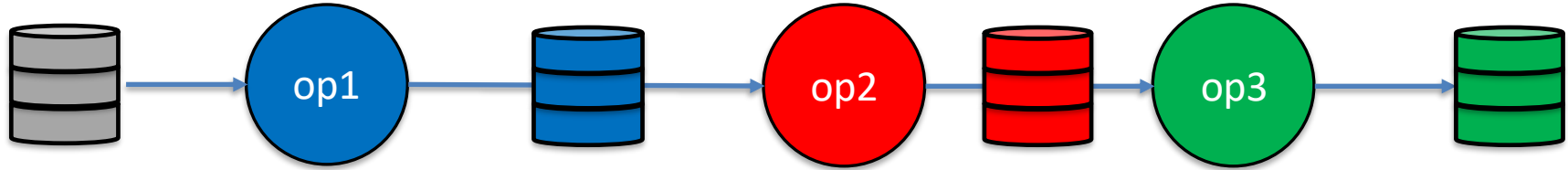
- Basic transformations:
- Cleaning
- Format revision
- Data threshold validation checks
- Restructuring
- Deduplication
- Advanced transformations:
- Filtering
- Merging
- Splitting
- Derivation
- Summarization
- Integration
- Aggregation
- Complex data validation

Load

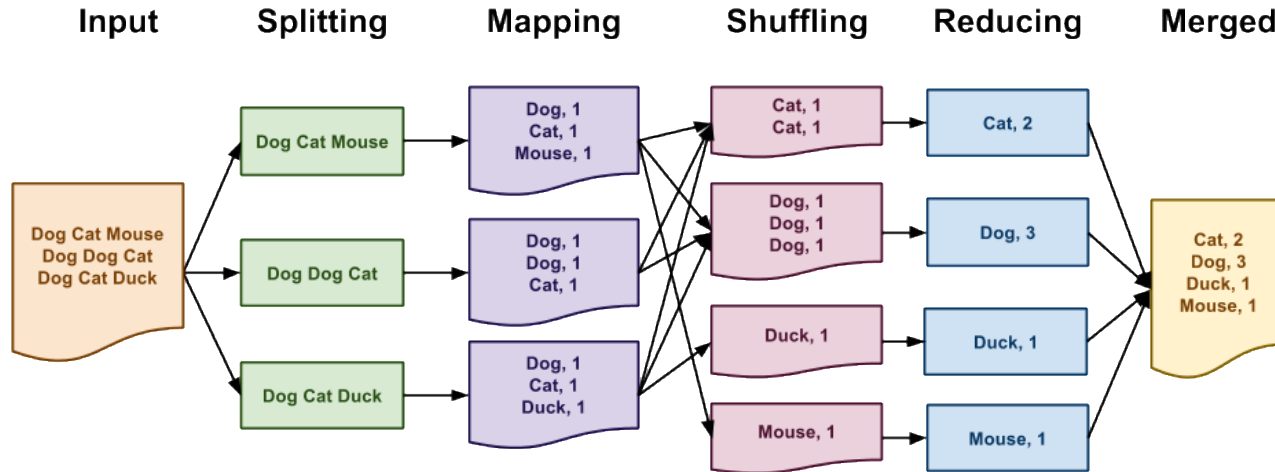
Finally, the load function is the process of writing converted data from a staging area to a target database, which may or may not have previously existed. Depending on the requirements of the application, this process may be either quite simple or intricate.

MapReduce

MapReduce is a data flow program with relatively simple operators on the data set.



With each operator implemented in parallel on multiple nodes for performance.



What we want more complex “operators?”

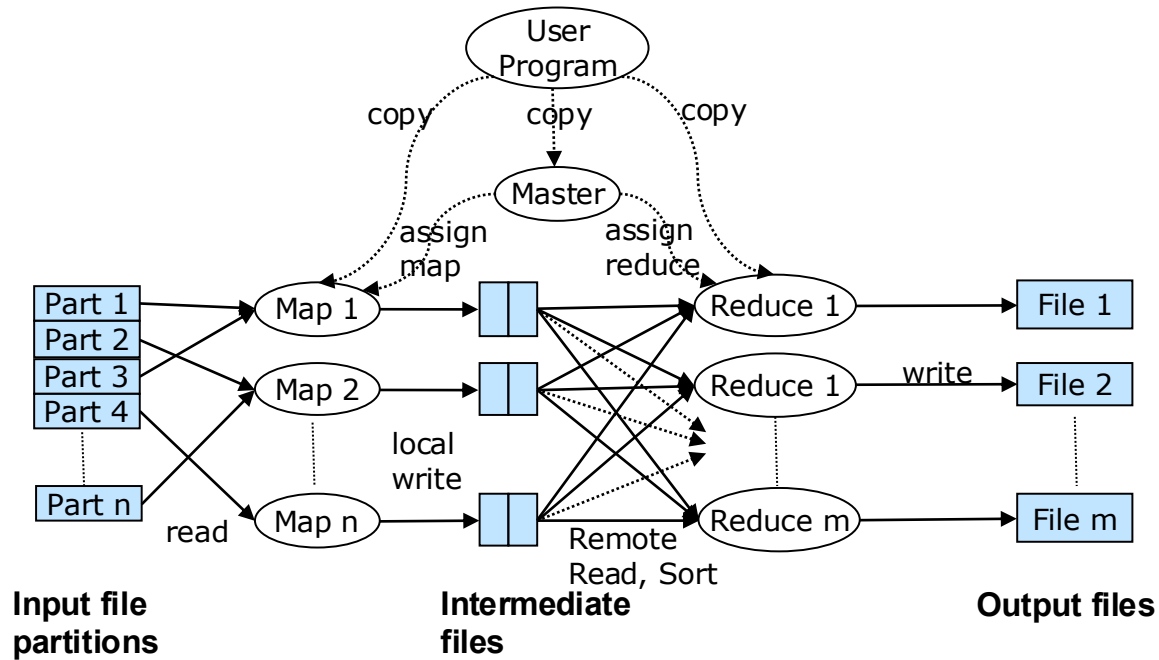


Map Reduce vs. Databases

- Map Reduce widely used for parallel processing
 - Google, Yahoo, and 100' s of other companies
 - Example uses: compute PageRank, build keyword indices, do data analysis of web click logs,
 - Allows procedural code in map and reduce functions
 - Allows data of any type
- Many real-world uses of MapReduce cannot be expressed in SQL
- But many computations are much easier to express in SQL
 - Map Reduce is cumbersome for writing simple queries



Parallel Processing of MapReduce Job





Map Reduce vs. Databases (Cont.)

- Relational operations (select, project, join, aggregation, etc.) can be expressed using Map Reduce
- SQL queries can be translated into Map Reduce infrastructure for execution
 - Apache Hive SQL, Apache Pig Latin, Microsoft SCOPE
- Current generation execution engines support not only Map Reduce, but also other algebraic operations such as joins, aggregation, etc. natively.

The Value of Blocks

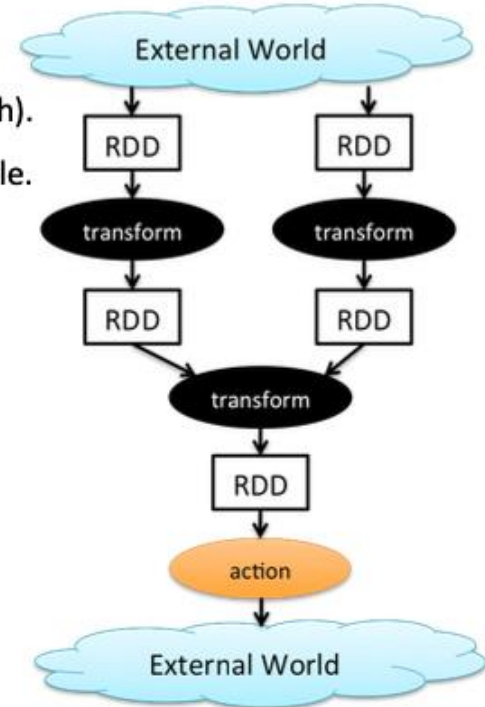
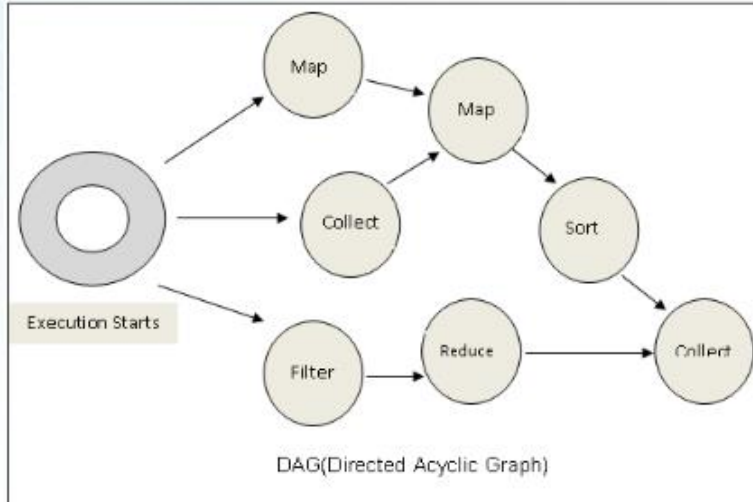
- The “normal” file system abstraction is a stream.
- Consider a very large CSV file:
 - We can read using the normal file system functions and APIs,
 - But we have no idea where records start/end, and
 - Which records on on which blocks.
 - ➔
 - Parallel processing of data in a file is very hard using the stream model.
- Databases, big data tools, etc. surface concepts of record size, block I/Os, mapping of and organization of records to blocks, block location, ➔
 - Can exploit multiple processors to parallel process large datasets.
 - Organize the data to optimize processing, file I/Os, etc.

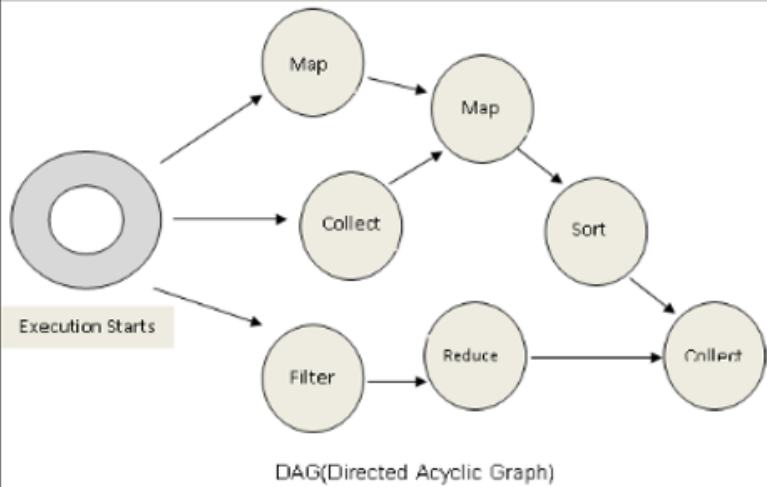
Algebraic Operations

Algebraic Operations

- Current generation execution engines
 - Natively support algebraic operations such as joins, aggregation, etc. natively.
 - Allow users to create their **own algebraic operators**
 - Support trees of algebraic operators that can be executed on **multiple nodes in parallel**
- E.g. Apache Tez, Spark
 - Tez provides low level API; Hive on Tez compiles SQL to Tez
 - Spark provides more user-friendly API
- In the relational model,
 - They are relations.
 - A fixed set of operators that produce relations from relations (closed).
 - Are declarative languages and compute the execution graph/plan.
- The Spark/... engines:
 - RDDs, Data Frames
 - Enable programmers to develop and add new operators.
 - Operators convert reliable distributed datasets/data frames to new RDDs/data frames.
 - Data engineer explicitly defines the execution graph (data flow).

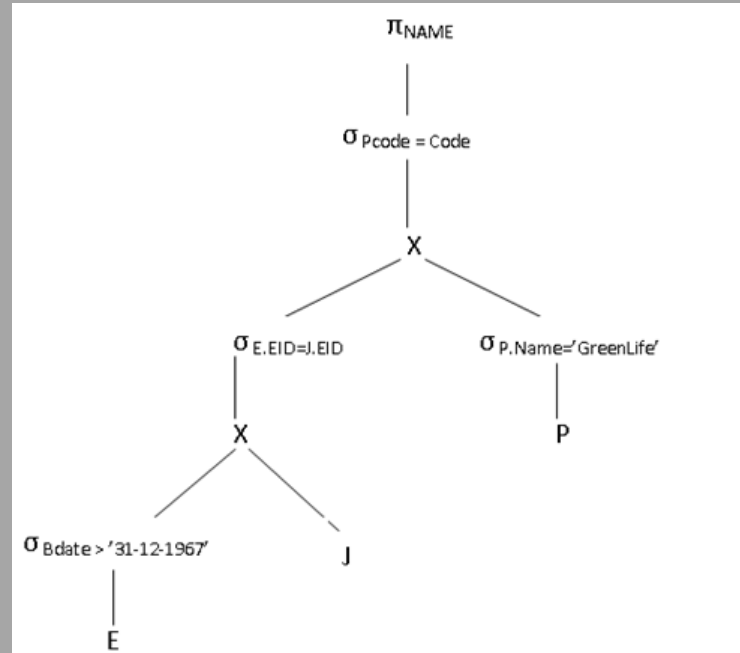
- All jobs in spark comprise a series of operators and run on a set of data.
- All the operators in a job are used to construct a DAG (Directed Acyclic Graph).
- The DAG is optimized by rearranging and combining operators where possible.





Conceptually similar to query evaluation graph, but ...

- You explicitly define the graph.
- You can develop your own operators.



Algebraic Operations in Spark

- **Resilient Distributed Dataset (RDD)** abstraction
 - Collection of records that can be stored across multiple machines
- RDDs can be created by applying algebraic operations on other RDDs
- RDDs can be lazily computed when needed
- Spark programs can be written in Java/Scala/R
 - Our examples are in Java
- Spark makes use of Java 8 Lambda expressions; the code
`s -> Arrays.asList(s.split(" ")).iterator()`
defines unnamed function that takes argument `s` and executes the expression `Arrays.asList(s.split(" ")).iterator()` on the argument
- Lambda functions are particularly convenient as arguments to map, reduce and other functions

DataFrame

edureka!

Inspired by DataFrames in R and Python (Pandas).

DataFrames API is designed to make big data processing on tabular data easier.

DataFrame is a distributed collection of data organized into named columns.

Provides operations to filter, group, or compute aggregates, and can be used with Spark SQL.

Can be constructed from structured data files, existing RDDs, tables in Hive, or external databases.

1. DataFrame in PySpark: Overview

In Apache Spark, a DataFrame is a distributed collection of rows under named columns. In simple terms, it is same as a table in relational database or an Excel sheet with Column headers. It also shares some common characteristics with RDD:

- **Immutable in nature** : We can create DataFrame / RDD once but can't change it. And we can transform a DataFrame / RDD after applying transformations.
- **Lazy Evaluations**: Which means that a task is not executed until an action is performed.
- **Distributed**: RDD and DataFrame both are distributed in nature.

My first exposure to DataFrames was when I learnt about Pandas. Today, it is difficult for me to run my data science workflow with out Pandas DataFrames. So, when I saw similar functionality in Apache Spark, I was excited about the possibilities it opens up!

<https://www.analyticsvidhya.com/blog/2016/10/spark-dataframe-and-operations/>

DataFrame features

edureka!

Ability to scale from KBs to PBs

Support for a wide array of data formats and storage systems

State-of-the-art optimization and code generation through the spark SQL catalyst optimizer

Seamless integration with all big data tooling and infrastructure via spark

APIs for Python, Java, Scala, and R

Ways to Create DataFrame in Spark

Hive Data

Csv Data

Json Data

RDBMS Data

XML Data

Parquet Data

Cassandra Data

RDDs

Spark SQL

DataFrame

	Col1	Col2	Col3	
Row 1				
Row 2				
Row 3				
.				

Wrap Up